



Licenciatura Ciências de Computação
Mestrado Integrado Eng^a. Informática
Mestrado Integrado Eng^a. Física

2020/21

A.J.Proença

Tema

Avaliação de Desempenho (IA-32)

Avaliação de Desempenho no IA-32 (1)



Estrutura do tema Avaliação de Desempenho (IA-32)

1. A avaliação de sistemas de computação
2. Técnicas de otimização de código (IM)
3. Técnicas de otimização de *hardware*
4. Técnicas de otimização de código (DM)
5. Outras técnicas de otimização
6. Medição de tempos ...

Análise do desempenho na execução de aplicações (1)



"Análise do desempenho": para quê?

- para avaliar Sistemas de Computação
 - identificação de métricas
 - latência, velocidade, ...
 - ligação entre métricas e fatores na arquitetura que influenciam o desempenho de um PU/núcleo

$$\text{Core}_{exec_time} = N^{\circ}_{instr} * CPI * T_{clock}$$

e . . .

- ... **construi-los mais rápidos**
- ... **melhorar a eficiência de execução de app's**

$$Core_{exec_time} = N^{\circ}_{instr} * CPI * T_{clock}$$



Análise dos componentes da fórmula:

- **Core_{exec_time}**
 - tempo de execução na PU/core, inclui acessos à memória, ...
- **N^o_{instr}**
 - efetivamente executadas; depende essencialmente de:
 - eficiência do compilador
 - do *instruction set*
- **CPI (Clock-cycles Per Instruction)**
 - tempo médio de exec de 1 instr, em ciclos; depende essencial/
 - complexidade da instrução (e acessos à memória ...)
 - paralelismo na execução de instruções
- **T_{clock}**
 - período do *clock*; depende essencialmente de:
 - complexidade da instrução e/ou sua subdivisão (*pipeline*)
 - microeletrónica

Análise do desempenho na execução de aplicações (2)



"Análise do desempenho": para quê?

- ... melhorar a eficiência de execução de app's
 - análise de técnicas de otimização do código
 - algoritmo / **codificação** / **compilação** / *assembly*
 - compromisso entre legibilidade e eficiência...
 - potencialidades e limitações dos compiladores...
 - técnicas independentes / dependentes da máquina
 - uso de *code profilers*
 - técnicas de medição de tempos
 - escala microscópica / macroscópica
 - uso de *cycle counters* / *interval counting*
 - métodos de medição confiável de tempos de execução

Potencialidades e limitações dos compiladores (1)



– um compilador moderno já inclui técnicas que

- exploram oportunidades para simplificar expressões
- usam um único cálculo de expressão em vários locais
- reduzem o nº de vezes que um cálculo é efetuado
- tiram partido de algoritmos sofisticados para
 - alocação eficiente dos registos
 - seleção e ordenação de código
- ... **mas** está limitado por certos fatores, tais como
 - nunca modificar o comportamento correto do programa
 - limitado conhecimento do programa e seu contexto
 - necessidade de ser rápido!

– e certas otimizações estão-lhe vedadas...

Potencialidades e limitações dos compiladores (2)



– exemplos de otimizações vedadas aos compiladores:

- pode trocar `twiddle1` por `twiddle2` ?

```
void twiddle1(int *xp,int *yp)
{
    *xp += *yp;
    *xp += *yp;
}
```

```
void twiddle2(int *xp,int *yp)
{
    *xp += 2* *yp;
}
```

teste: **xp** igual a **yp**; que acontece?

- pode trocar `func1` por `func2` ?

```
int f(int n)
int func1(x)
{
    return f(x)+f(x)+f(x)+f(x);
}
```

```
int f(int n)
int func2(x)
{
    return 4*f(x);
}
```

teste: e se `f` for...?

```
int counter = 0;

int f(int x)
{
    return counter++;
}
```

Avaliação de Desempenho no IA-32 (2)



Estrutura do tema Avaliação de Desempenho (IA-32)

1. A avaliação de sistemas de computação
2. Técnicas de otimização de código (IM)
3. Técnicas de otimização de *hardware*
4. Técnicas de otimização de código (DM)
5. Outras técnicas de otimização
6. Medição de tempos

Otimização de código: técnicas independentes da máquina



***"Independentes da máquina": aplicam-se
a qualquer processador / compilador***

Algumas técnicas de otimização:

- movimentação de código
 - reduzir frequência de execução (compiladores têm limitações)
- simplificação de cálculos
 - substituir operações por outras mais simples
- partilha de cálculos
 - identificar e explicitar subexpressões comuns

Metodologia a seguir:

- apresentação de alguns conceitos
- análise de um programa exemplo a otimizar
- introdução de uma técnica de medição de desempenho

Otimizações independentes da máquina: movimentação de código (1)



- **Movimentação de código**

- Objetivo: minimizar repetição de cálculos
 - se produzir sempre o mesmo resultado
 - especialmente retirar código do interior de ciclos
- **Exemplo:** definir uma matriz $a[n, n]$ em que todas as colunas são iguais e cada coluna é igual ao vetor $b[n]$

```
for (i = 0; i < n; i++)  
    for (j = 0; j < n; j++)  
        a[n*i + j] = b[j];
```



```
for (i = 0; i < n; i++) {  
    int ni = n*i;  
    for (j = 0; j < n; j++)  
        a[ni + j] = b[j];  
}
```

Otimizações independentes da máquina: movimentação de código (2)



- A maioria dos compiladores é eficiente a lidar com código com *arrays* e estruturas simples com ciclos
- Código gerado pelo GCC:

```
for (i = 0; i < n; i++)  
    for (j = 0; j < n; j++)  
        a[n*i + j] = b[j];
```

```
for (i = 0; i < n; i++) {  
    int ni = n*i;  
    int *p = a+ni;  
    for (j = 0; j < n; j++)  
        *p++ = b[j];  
}
```

```
imull %ebx,%eax      # i*n colocado em %eax  
movl 8(%ebp),%edi    # apont p/ array a em %edi  
leal (%edi,%eax,4),%edx # p=a+n*i (ajustado 4*) em %edx  
.L40:               # Ciclo interior  
movl 12(%ebp),%edi   # apont p/ array b em %edi  
movl (%edi,%ecx,4),%eax # b+j (ajustado 4*) em %eax  
movl %eax, (%edx)    # *p=b[j] (%edx aponta para b+j)  
addl $4,%edx         # p++ (ajustado 4*)  
incl %ecx            # j++  
jl .L40             # loop if j<n
```

Otimizações independentes da máquina: simplificação de cálculos



- Substituir operações “caras” por outras +simples
 - *shift* ou *add* em vez de *mul* ou *div* por potências de 2
 - $16 * x \rightarrow x \ll 4$
 - escolha pode ser dependente da máquina
 - reconhecer sequência de produtos

Já visto!

```
for (i = 0; i < n; i++)  
    for (j = 0; j < n; j++)  
        a[n*i + j] = b[j];
```



```
int ni = 0;  
for (i = 0; i < n; i++) {  
    int ni = n*i;  
    for (j = 0; j < n; j++)  
        *p++ = b[j];  
    ni += n;  
}
```

Otimizações independentes da máquina: partilha de cálculos



- **Partilhar sub-expressões comuns**
 - reutilizar partes de expressões
 - compiladores não são particularmente famosos a explorar propriedades aritméticas

```
/* Soma vizinhos de i,j */  
up =    val[(i-1)*n + j];  
down =  val[(i+1)*n + j];  
left =  val[i*n    + j-1];  
right = val[i*n    + j+1];  
sum = up + down + left + right;
```

3 multiplicações: $i*n$, $(i-1)*n$, $(i+1)*n$

```
leal -1(%edx),%ecx    # i-1  
imull %ebx,%ecx       # (i-1)*n  
leal 1(%edx),%eax     # i+1  
imull %ebx,%eax       # (i+1)*n  
imull %ebx,%edx       # i*n
```

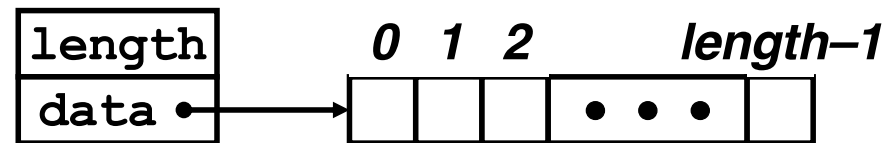
```
int inj = i*n + j;  
up =    val[inj - n];  
down =  val[inj + n];  
left =  val[inj - 1];  
right = val[inj + 1];  
sum = up + down + left + right;
```

1 multiplicação: $i*n$

Análise detalhada de um exemplo: introdução ao Abstract Data Type (ADT)



O vetor ADT:



- **Funções associadas**

```
vec_ptr new_vec(int len)
```

- cria vetor com o comprimento especificado

```
int get_vec_element(vec_ptr v, int index, int *dest)
```

- recolhe um elemento do vetor e guarda-o em *dest
- devolve 0 se fora de limites, 1 se obtido com sucesso

```
int *get_vec_start(vec_ptr v)
```

- devolve apontador para início dos dados do vetor

- Idêntico às implementações de *arrays* em Pascal, ML, Java
 - i.e., faz sempre verificação de limites (*bounds*)

Análise detalhada de um exemplo: o procedimento a otimizar (1)



```
void combine1(vec_ptr v, int *dest)
{
    int i;
    *dest = 0;
    for (i = 0; i < vec_length(v); i++) {
        int val;
        get_vec_element(v, i, &val);
        *dest += val;
    }
}
```

- **Procedimento**
 - calcula a soma de todos os elementos do vetor
 - guarda o resultado numa localização “destino”
 - estrutura e operações do vetor definidos via ADT
- **Tempos de execução: que/como medir?**

Análise detalhada de um exemplo: tempos de execução (1)



```
void combinel(vec_ptr v, int *dest)
{
    int i;
    *dest = 0;
    for (i = 0; i < vec_length(v); i++) {
        int val;
        get_vec_element(v, i, &val);
        *dest += val;
    }
}
```

Tempos de execução: que medir e como medir?

- **que medir**: em programas iterativos (com ciclos), uma medida útil é a duração da operação para cada um dos elementos da iteração:
 - **ciclos** (de *clock*) **por elemento, CPE**
- **como medir o CPE**: fazer várias medições de tempo para dimensões variáveis de ciclos, e calculá-lo através do traçado gráfico; o CPE é o declive da reta *best fit*, obtida pelo método dos mínimos quadrados
 - análise gráfica de um exemplo...

Análise detalhada de um exemplo: tempos de execução (2)



```
void vsum1(int n)
{
    int i;
    for (i=0; i<n; i++)
        c[i] = a[i] + b[i];
}
```

```
void vsum2(int n)
{
    int i;
    for (i=0; i<n; i+=2){
        c[i]  = a[i]  + b[i];
        c[i+1]= a[i+1]+ b[i+1];
    }
}
```

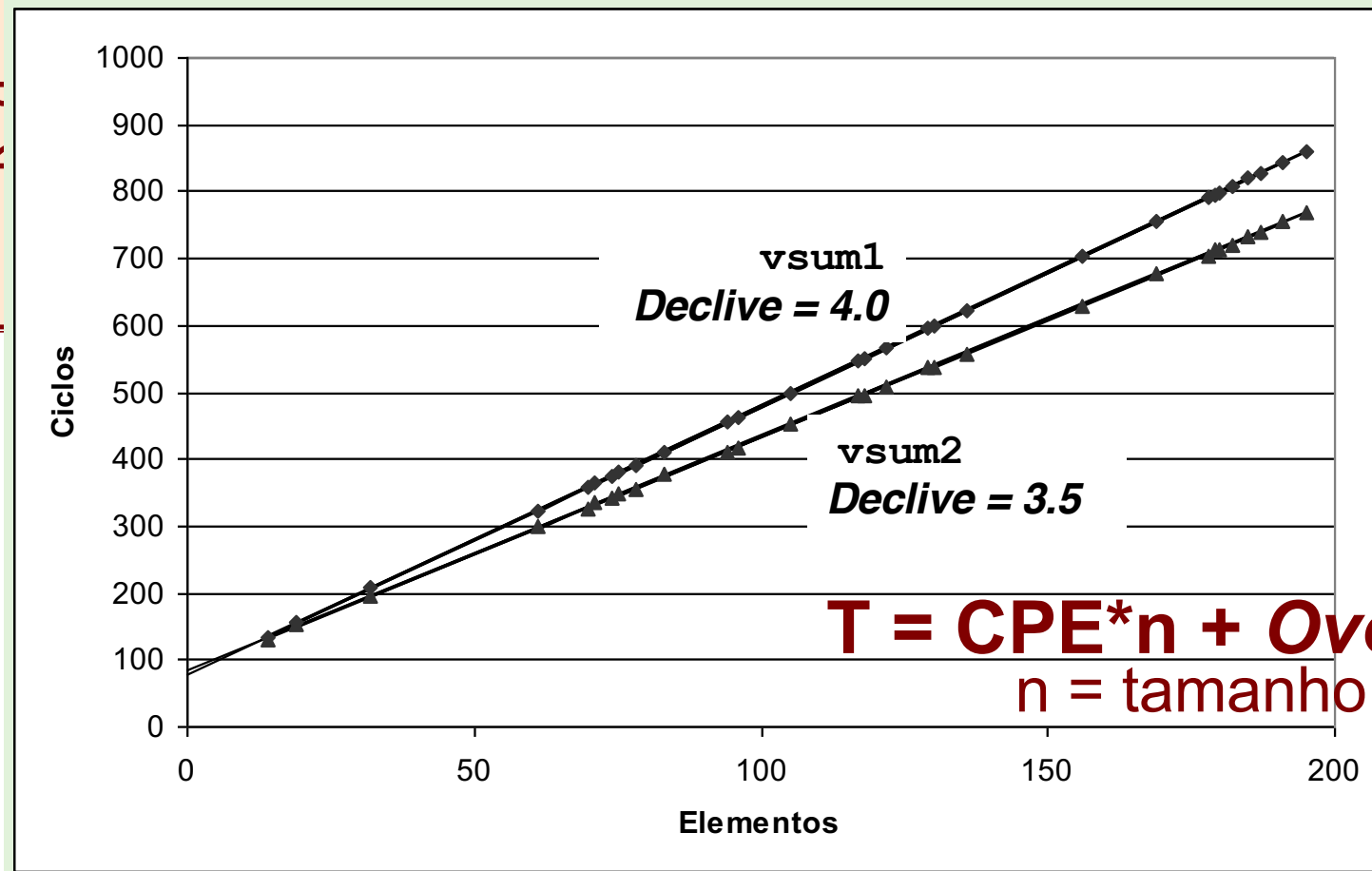
Análise detalhada de um exemplo: tempos de execução (3)



```
void vsum1(int n)
{
    int
    for
}
```

```
void vsum2(int n)
```

```
{
    [i];
    [i+1];
}
```



$$T = CPE * n + \text{Overhead}$$

n = tamanho do vetor

Análise detalhada de um exemplo: o procedimento a otimizar (2)



```
void combine1(vec_ptr v, int *dest)
{
    int i;
    *dest = 0;
    for (i = 0; i < vec_length(v); i++) {
        int val;
        get_vec_element(v, i, &val);
        *dest += val;
    }
}
```

- **Procedimento**
 - calcula a soma de todos os elementos do vetor
 - guarda o resultado numa localização destino
 - estrutura e operações do vetor definidos via ADT
- **Tempo de execução (inteiros) :**
 - compilado sem qq otimização: 42.06 CPE
 - compilado com **-O2**: 31.25 CPE

Análise detalhada do exemplo: à procura de ineficiências...



Versão goto

```
void combine1-goto(vec_ptr v, int *dest)
{
    int i = 0;
    int val;
    *dest = 0;
    if (i >= vec_length(v)) goto done;
loop:
    get_vec_element(v, i, &val);
    *dest += val;
    i++;
    if (i < vec_length(v))
        goto loop
done:
}
```

1 iteração

Ineficiência óbvia:

- função `vec_length` invocada em cada iteração
- ... mesmo sendo para calcular o mesmo valor!

Análise detalhada do exemplo: movimentação de código



Otimização 1:

- mover invocação de `vec_length` para fora do ciclo interior
 - o valor não altera de iteração para iteração
- **CPE:** de 31.25 para **20.66** (compilado com `-O2`)
 - `vec_length` impõe um *overhead* constante, mas significativo

```
void combine2(vec_ptr v, int *dest)
{
    int i;
    int length = vec_length(v);
    *dest = 0;
    for (i = 0; i < length; i++) {
        int val;
        get_vec_element(v, i, &val);
        *dest += val;
    }
}
```

Bloqueadores de otimização: a invocação de procedimentos/funções



Por que razão o compilador não moveu `vec_1en` para fora do ciclo?

- a função pode ter efeitos colaterais
 - por ex., alterar o estado global de cada vez que é invocada
- a função poderá não devolver os mesmos valores consoante o arg
 - depende de outras partes do estado global

Por que razão o compilador não analisou o código de `vec_1en`?

- otimização interprocedimental não é usada extensivamente devido ao seu elevado custo

Aviso:

- o compilador trata invocação de procedimentos como uma *black box*
- as otimizações são pobres em redor de invoc de procedimentos

Análise detalhada do exemplo: simplificação de cálculos



Otimização 2:

- evitar invocação de `get_vec_element` para ir buscar cada elemento do vetor
 - obter apontador para início do *array* antes do ciclo
 - dentro do ciclo trabalhar apenas com o apontador
- **CPE:** de 20.66 para **6.00** (compilado com **-O2**)
 - invocação de funções é dispendioso, mas tem riscos dispensá-lo
 - validação de limites de *arrays* é dispendioso

```
void combine3(vec_ptr v, int *dest)
{
    int i;
    int length = vec_length(v);
    int *data = get_vec_start(v);
    *dest = 0;
    for (i = 0; i < length; i++) {
        *dest += data[i];
    }
}
```

Análise detalhada do exemplo: eliminar referências desnecessárias à memória



Otimização 3:

- não é preciso guardar resultado em `dest` a meio dos cálculos
 - adiciona-se a variável local `sum` que é alocada a um registo
 - poupam-se 2 acessos à memória por ciclo (1 leitura + 1 escrita)
- **CPE:** de 6.00 para **2.00** (compilado com **-O2**)
 - acessos à memória são dispendiosos

```
void combine4(vec_ptr v, int *dest)
{
    int i;
    int length = vec_length(v);
    int *data = get_vec_start(v);
    int sum = 0;
    for (i = 0; i < length; i++)
        sum += data[i];
    *dest = sum;
}
```

Análise detalhada do exemplo: como detetar referências desnecessárias à memória



Combine3

```
.L18:  
    movl (%ecx,%edx,4),%eax  
    addl %eax, (%edi)  
    incl %edx  
    cmpl %esi,%edx  
    jl .L18
```

Combine4

```
.L24:  
    addl (%eax,%edx,4),%ecx  
  
    incl %edx  
    cmpl %esi,%edx  
    jl .L24
```

Desempenho comparativo

– Combine3

- 5 instruções em 6 ciclos de *clock*
- `addl` tem de ler e escrever na memória em cada iteração

– Combine4

- 4 instruções em 2 ciclos de *clock*

Bloqueadores de otimização: aliasing de memória



- **Aliasing**

- 2 referências distintas à memória especificam a mesma localização

- **Example**

– v: [3, 2, 17] ***dest**
– combine3(v, get_vec_start(v)+2) --> ?
– combine4(v, get_vec_start(v)+2) --> ?

- **Observações**

- fácil de acontecer em C, porque esta linguagem permite
 - operações aritméticas com endereços
 - acesso direto a valores armazenados em estruturas de dados
- criar o hábito de usar variáveis locais
 - para acumular resultados dentro de ciclos
 - **como forma de avisar o compilador para não se preocupar com *aliasing***

Análise detalhada do exemplo: forma genérica e abstrata de *combine*



```
void abstract_combine4(vec_ptr v, data_t *dest)
{
    int i;
    int length = vec_length(v);
    data_t *data = get_vec_start(v);
    data_t t = IDENT;
    for (i = 0; i < length; i++)
        t = t OP data[i];
    *dest = t;
}
```

Tipos de dados

- declarações distintas para **data_t**
 - int
 - float
 - double

Operações

- definições diferentes para **OP** e **IDENT**
 - + / 0
 - * / 1

Otimizações independentes da máquina: resultados experimentais com o programa exemplo



Otimizações

- reduzir invocação func e acessos à memória dentro do ciclo

Método	Inteiro		Real (prec simp)	
	+	*	+	*
<i>Abstract -g</i>	42.06	41.86	41.44	160.00
<i>Abstract -O2</i>	31.25	33.25	31.25	143.00
<i>Move vec_length</i>	20.66	21.25	21.15	135.00
<i>Acesso aos dados</i>	6.00	9.00	8.00	117.00
<i>Acum. em temp</i>	2.00	4.00	3.00	5.00

- Anomalia no desempenho
 - cálculos de produtos de FP excepcional/ lento com todos
 - aceleração considerável quando acumulou em `temp`
 - causa: unidade de FP do IA-32
 - memória usa formato com 64-bit, registo usa 80
 - os dados causaram *overflow* com 64 bits, mas não com 80

Avaliação de Desempenho no IA-32 (3)



Estrutura do tema Avaliação de Desempenho (IA-32)

1. A avaliação de sistemas de computação
2. Técnicas de otimização de código (IM)
3. **Técnicas de otimização de *hardware***
4. Técnicas de otimização de código (DM)
5. Outras técnicas de otimização
6. Medição de tempos ...

Eficiência em Sistemas de Computação: oportunidades para otimizar na arquitetura



Otimização do desempenho (no *h/w*)

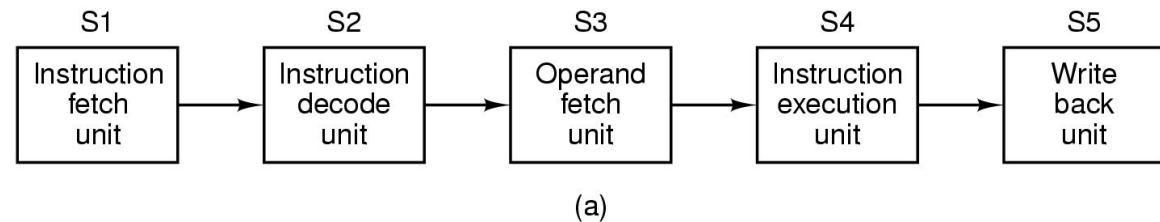
- no processador: com **paralelismo**
 - ao nível do processo (*multicore*/distribuídos/heterogéneos)
 - ao nível da instrução num core (*Instruction Level Parallelism*)
 - na execução do código:
 - » paralelismo desfasado (*pipeline*)
 - » paralelismo "real" (VLIW, superescalaridade)
 - paralelismo só nos dados (processamento vetorial)
- no acesso à memória e com **hierarquia de memória**
 - na transferência de informação de/para a memória
 - com paralelismo desfasado (*interleaving*)
 - com paralelismo "real" (>largura do *bus*, mais canais)
 - *cache* dedicada/partilhada, acesso UMA/NUMA...

Paralelismo no processador

Exemplo 1



Exemplo de pipeline

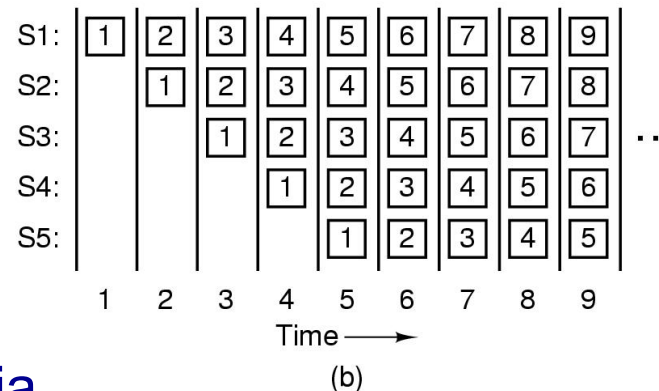


Objetivo

- CPI = 1

Problemas:

- dependências de dados
- latências nos acessos à memória
- saltos condicionais; propostas de solução para minimizar perdas:
 - executar sempre a instrução "que se segue"
 - usar o historial dos saltos anteriores (1 ou mais bits)
 - executar os 2 percursos alternativos até à tomada de decisão

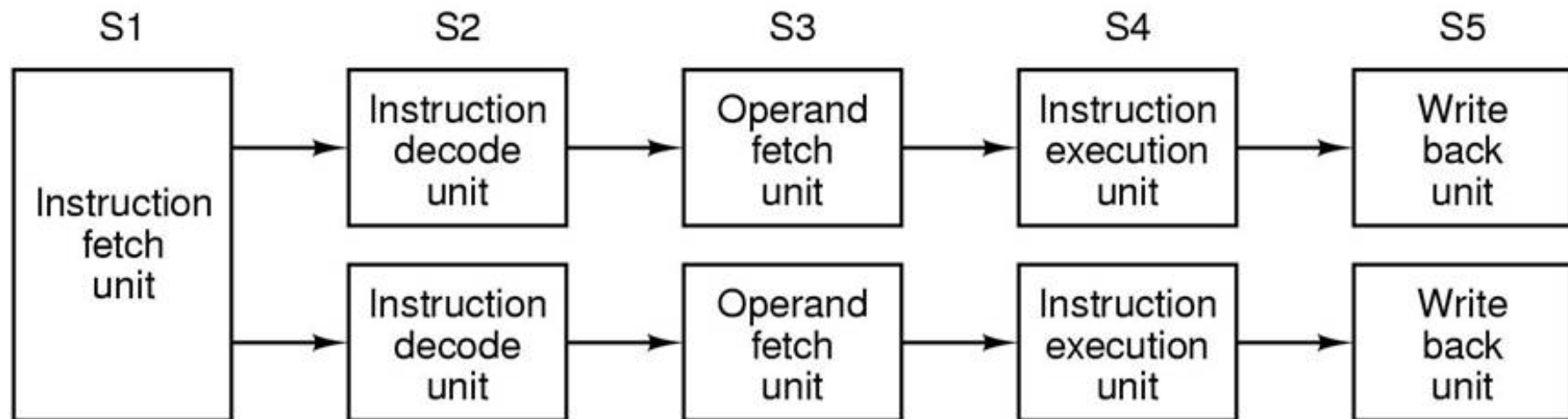


Paralelismo no processador

Exemplo 2

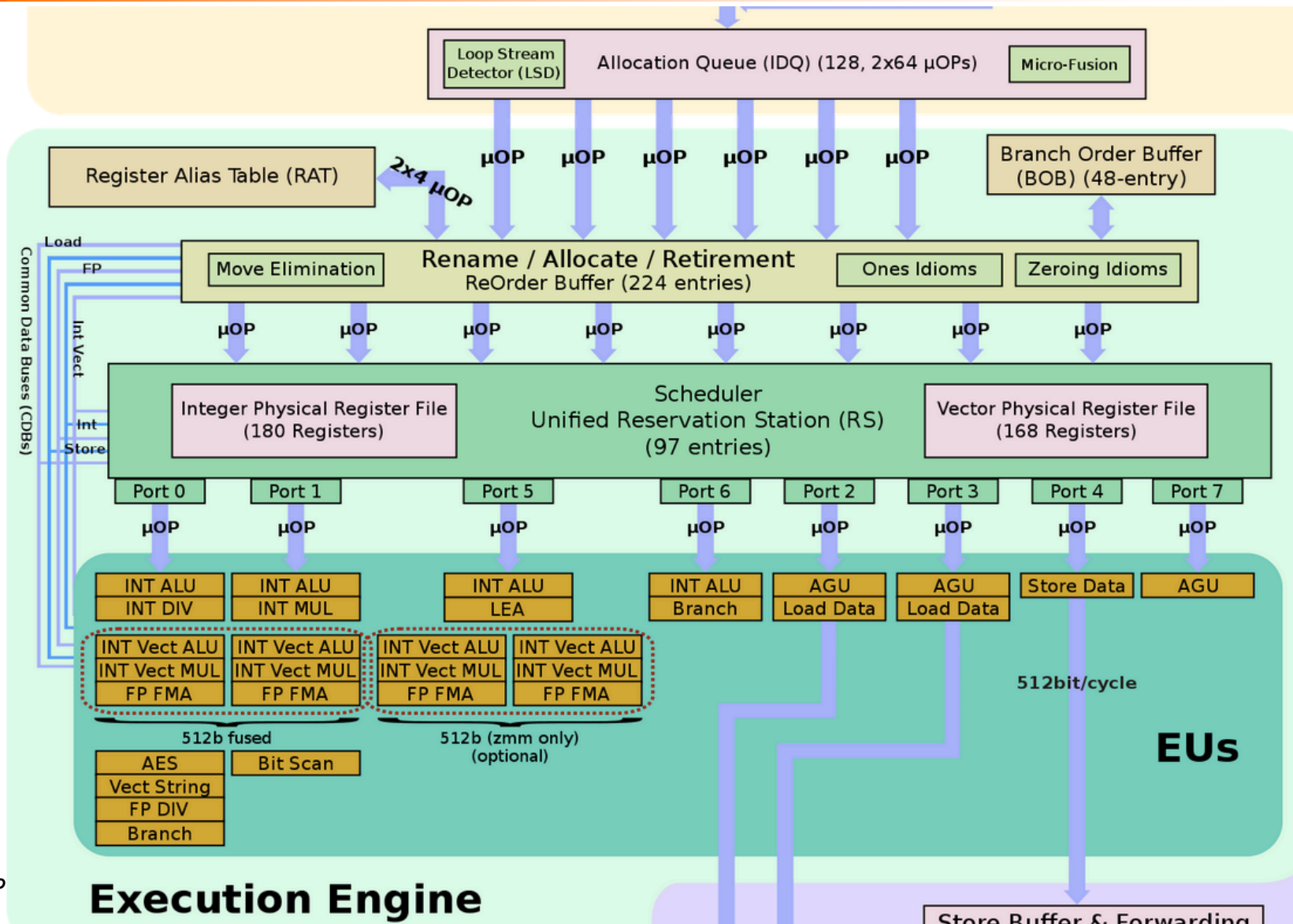


Exemplo de superescalaridade (2 vias)

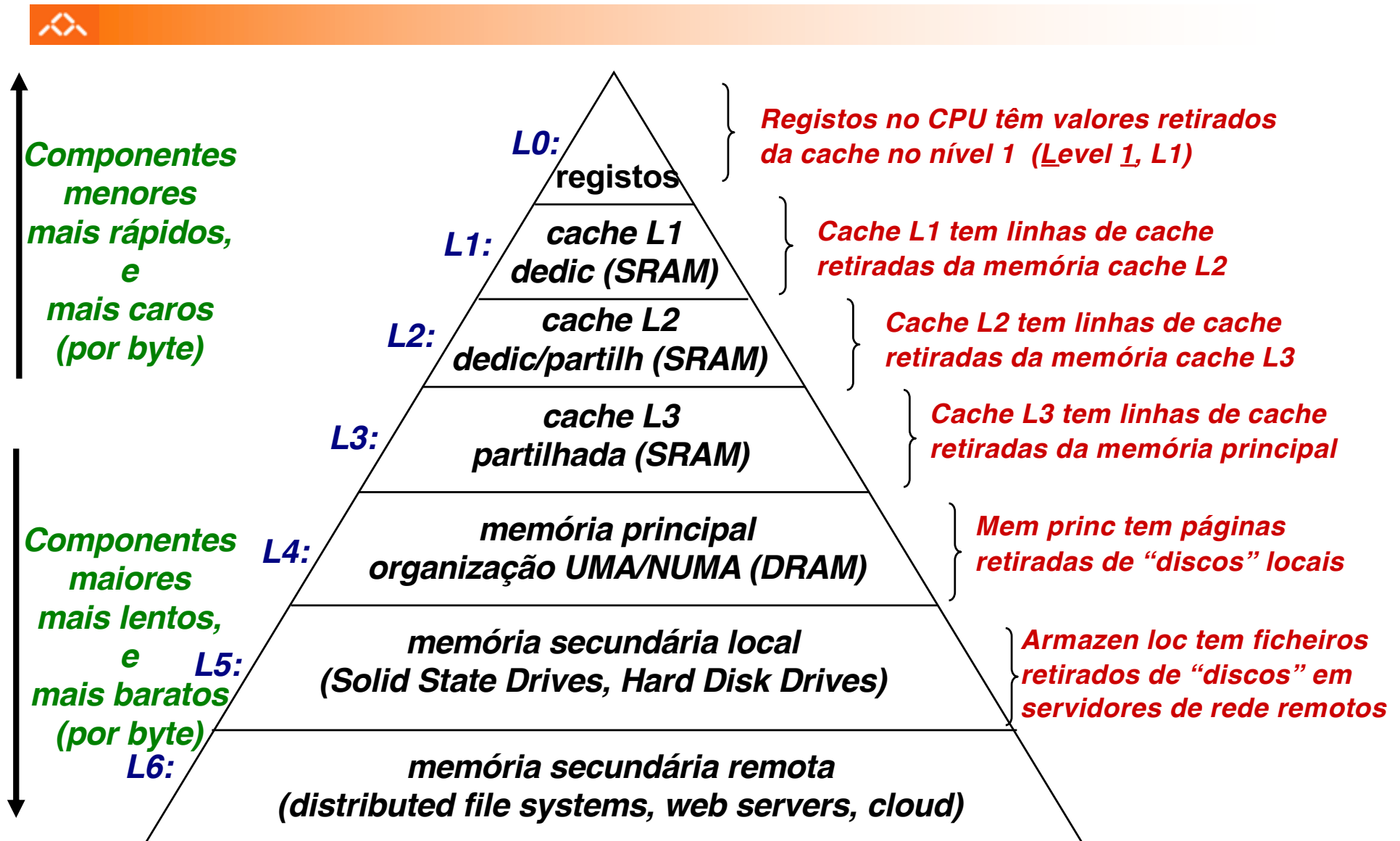


Paralelismo no processador

Exemplo 3 (superescalaridade 8-vias no Intel Skylake)



Organização hierárquica da memória



Sucesso da hierarquia de memória: o princípio da localidade



Princípio da Localidade:

- programas tendem a reusar dados e instruções próximos daqueles que foram recentemente usados ou referenciados por eles
- **Localidade Espacial:** itens em localizações contíguas tendem a ser referenciados em tempos próximos
- **Localidade Temporal:** itens recentemente referenciados serão provavelmente referenciados no futuro próximo

Exemplo da Localidade :

• Dados

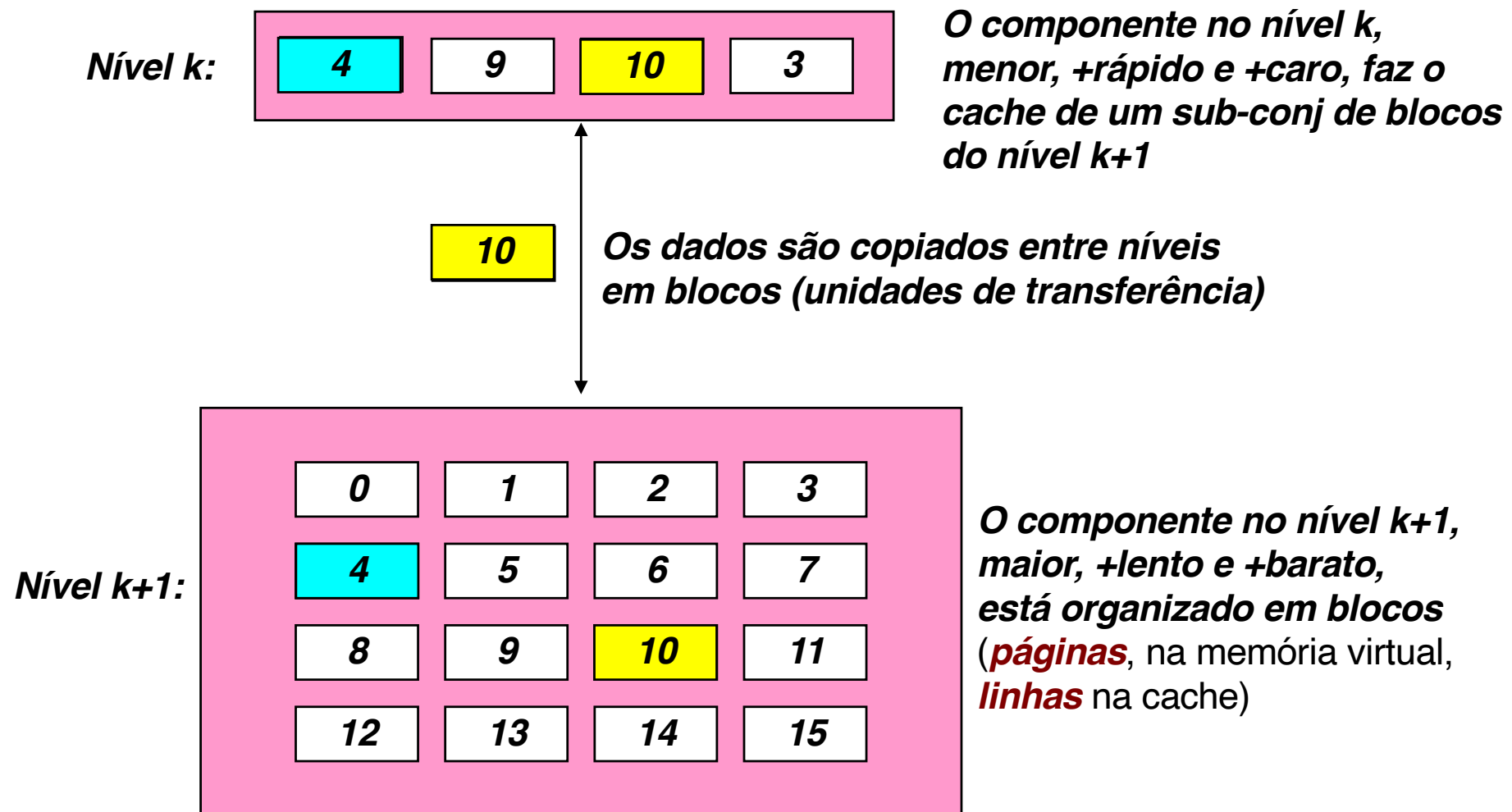
- os elementos do *array* são referenciados em instruções sucessivas: **Localidade Espacial**
- a variável `sum` é acedida em cada iteração: **Localidade Temporal**

• Instruções

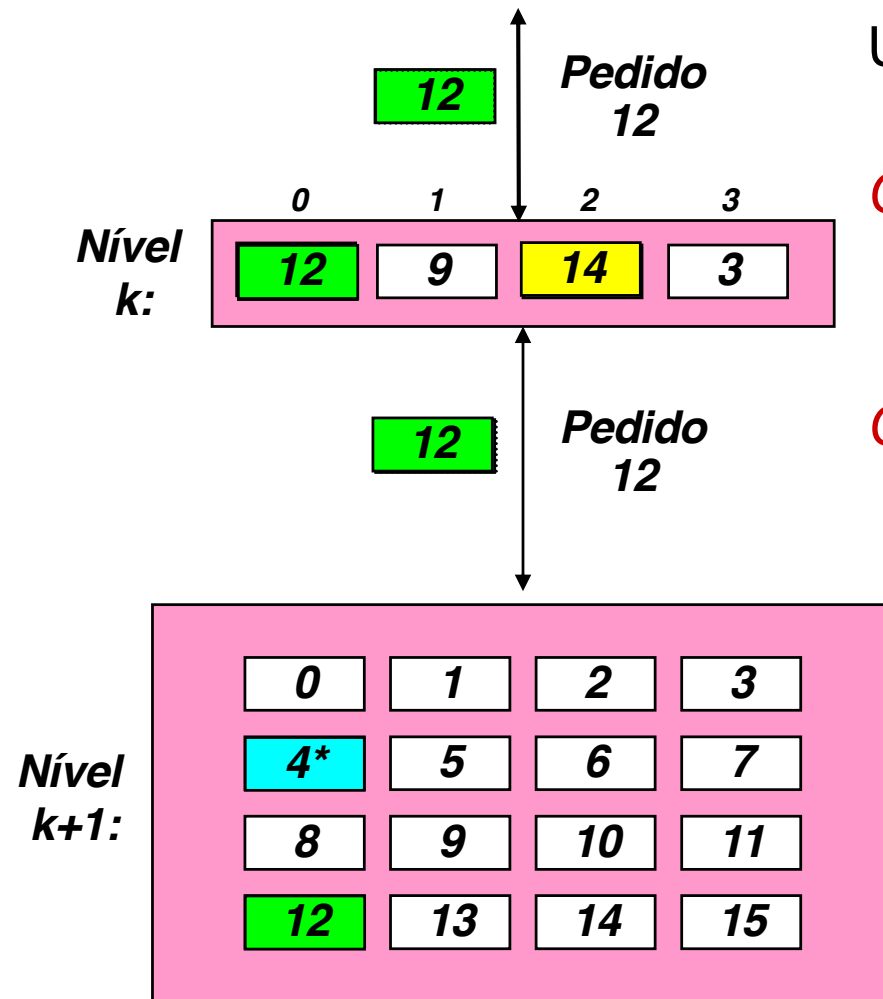
- as instruções são acedidas sequencialmente: **Localidade Espacial**
- o ciclo é repetidamente acedido: **Localidade Temporal**

```
sum = 0;  
for (i = 0; i < n; i++)  
    sum += a[i];  
return sum;
```

A cache numa hierarquia de memória: introdução



A cache numa hierarquia de memória: conceitos



Um programa pede pelo objeto d , que está armazenado num bloco b

Cache hit

- o programa encontra b na *cache* no nível k .

Por ex., bloco 14

Cache miss

- b não está no nível k , logo a *cache* do nível k deve buscá-lo do nível $k+1$.

Por ex., bloco 12

- se a *cache* do nível k está cheia, então um dos blocos deve ser substituído (retirado); qual?

- **Replacement policy:** que bloco deve ser retirado? Por ex., LRU

- **Placement policy:** onde colocar o novo bloco? Por ex., $b \bmod 4$

A cache numa hierarquia de memória: métricas de desempenho



Miss Rate

- percentagem de referências à memória que não tiveram sucesso na *cache* ($\#misses / \#acessos$)
- valores típicos:
 - 3-10% para L1
 - pode ser menor para L2 ($< 1\%$), dependendo do tamanho, etc.

Hit Time

- tempo para a *cache* entregar os dados ao processador (inclui o tempo para verificar se a linha está na *cache*)
- valores típicos :
 - 1-2 ciclos de *clock* para L1
 - 3-10 ciclos de *clock* para L2

Miss Penalty

- tempo extra necessário para ir buscar uma linha após *miss*
 - tipicamente 50-100 ciclos para aceder à memória principal

A cache numa hierarquia de memória: regras na codificação de programas



Referenciar repetidamente uma variável é positivo!

(localidade temporal)

Referenciar elementos consecutivos de um *array* é positivo!

(localidade espacial)

Exemplos:

– **cache fria, palavras de 4-bytes, blocos (linhas) de cache com 4-palavras**

```
int sum_array_rows(int a[M][N])
{
    int i, j, sum = 0;

    for (i = 0; i < M; i++)
        for (j = 0; j < N; j++)
            sum += a[i][j];
    return sum;
}
```

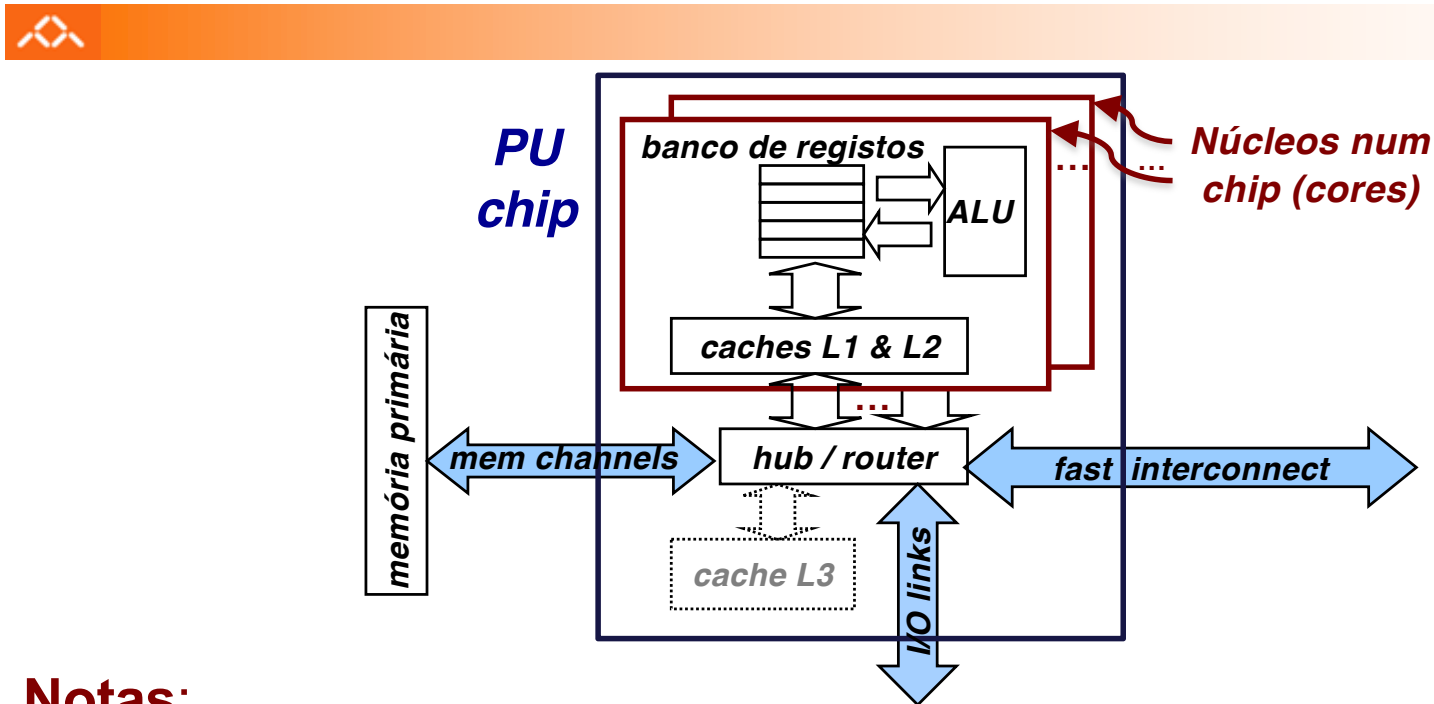
Miss rate = $1/4 = 25\%$

```
int sum_array_cols(int a[M][N])
{
    int i, j, sum = 0;

    for (j = 0; j < N; j++)
        for (i = 0; i < M; i++)
            sum += a[i][j];
    return sum;
}
```

Miss rate = até 100%

A cache em arquiteturas multicore



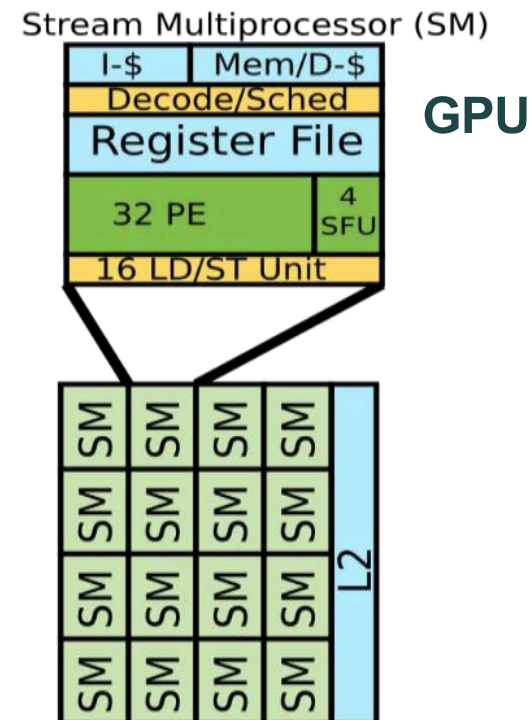
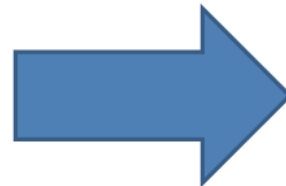
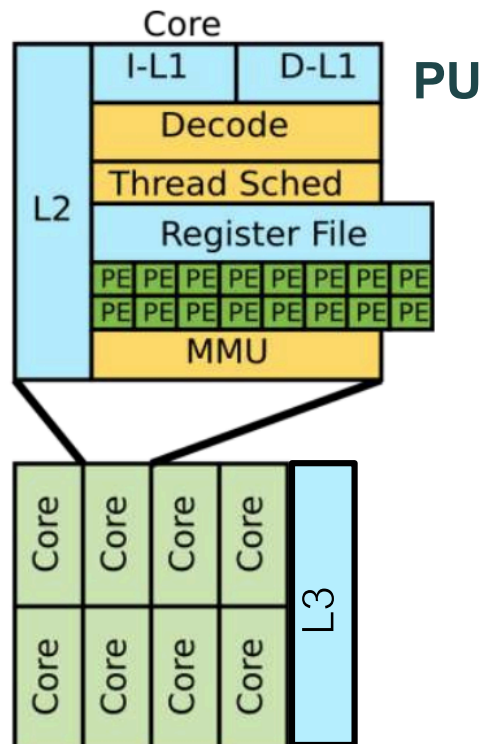
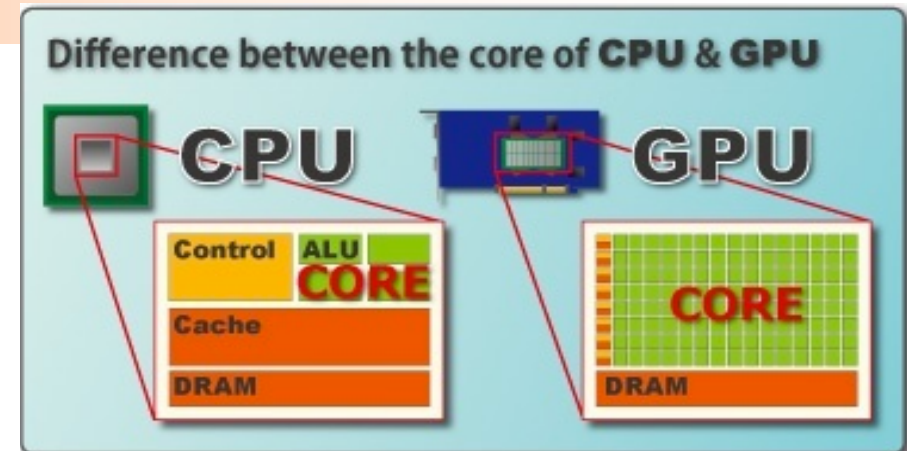
Notas:

- as **caches L1** de dados e de instruções são normalmente distintas
- as **caches L2** em *multicores* podem ser partilhadas por outras *cores*
- muitos *cores* partilhando uma única memória traz complexidades:
 - manutenção da coerência da informação nas *caches*
 - encaminhamento e partilha dos circuitos de acesso à memória

Evolução das arquiteturas multicore: dos PU genéricos aos PU em aceleradores



Ex. de acelerador de computação:
GPU



Paralelismo de dados: do SIMD em CPU ao SIMT em GPU



~~CPU~~

SIMD

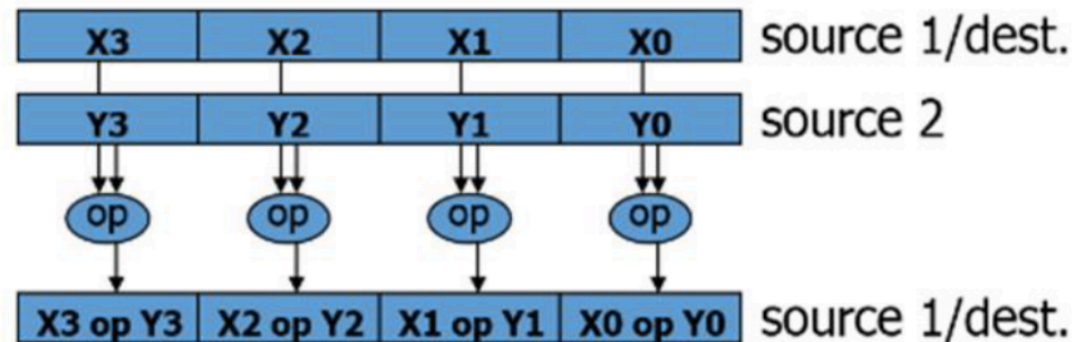
1 instruction – multiple data

SIMD:

Single Instruction Multiple Data

SSE2/3/4 – Neon – AltiVec

AVX – AVX2...



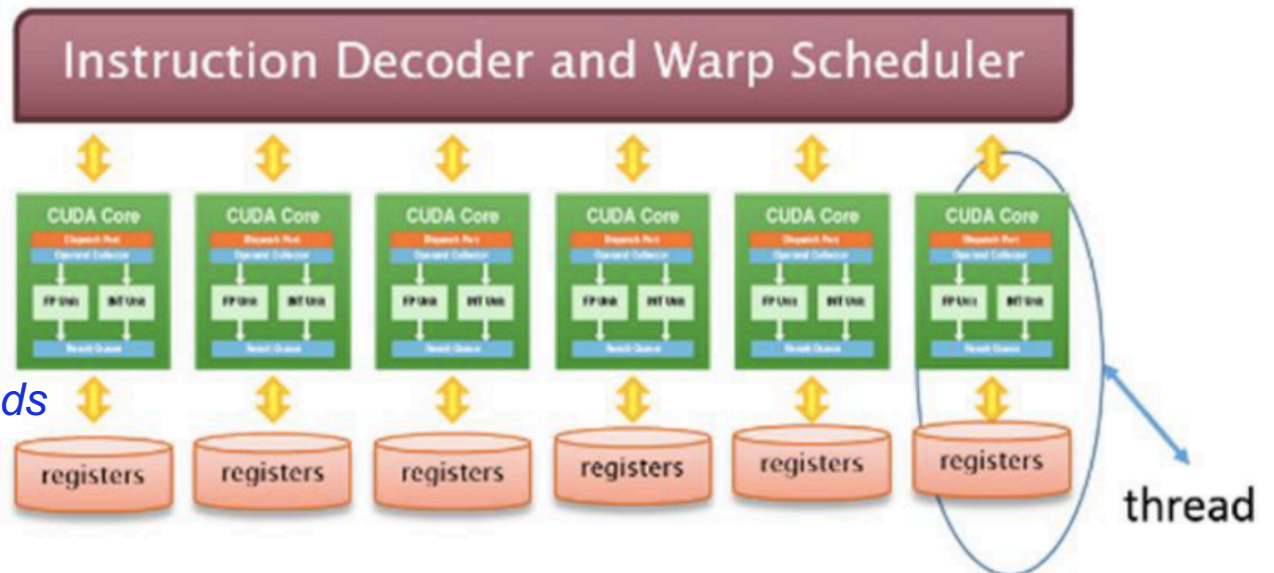
GPU

SIMT

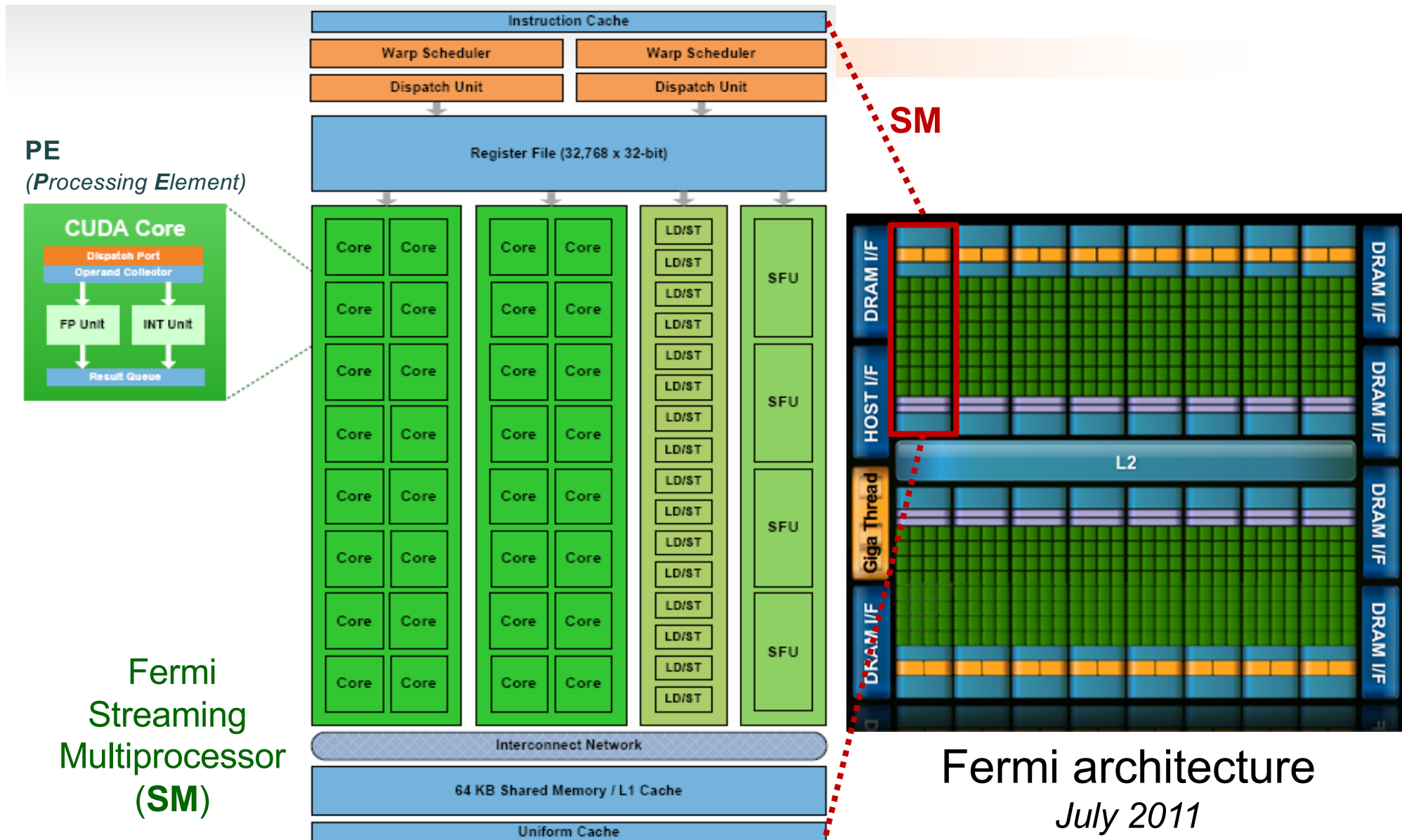
1 instruction – multiple threads

SIMT:

Single Instruction Multiple Threads



A arquitetura dos GPUs Fermi da NVidia

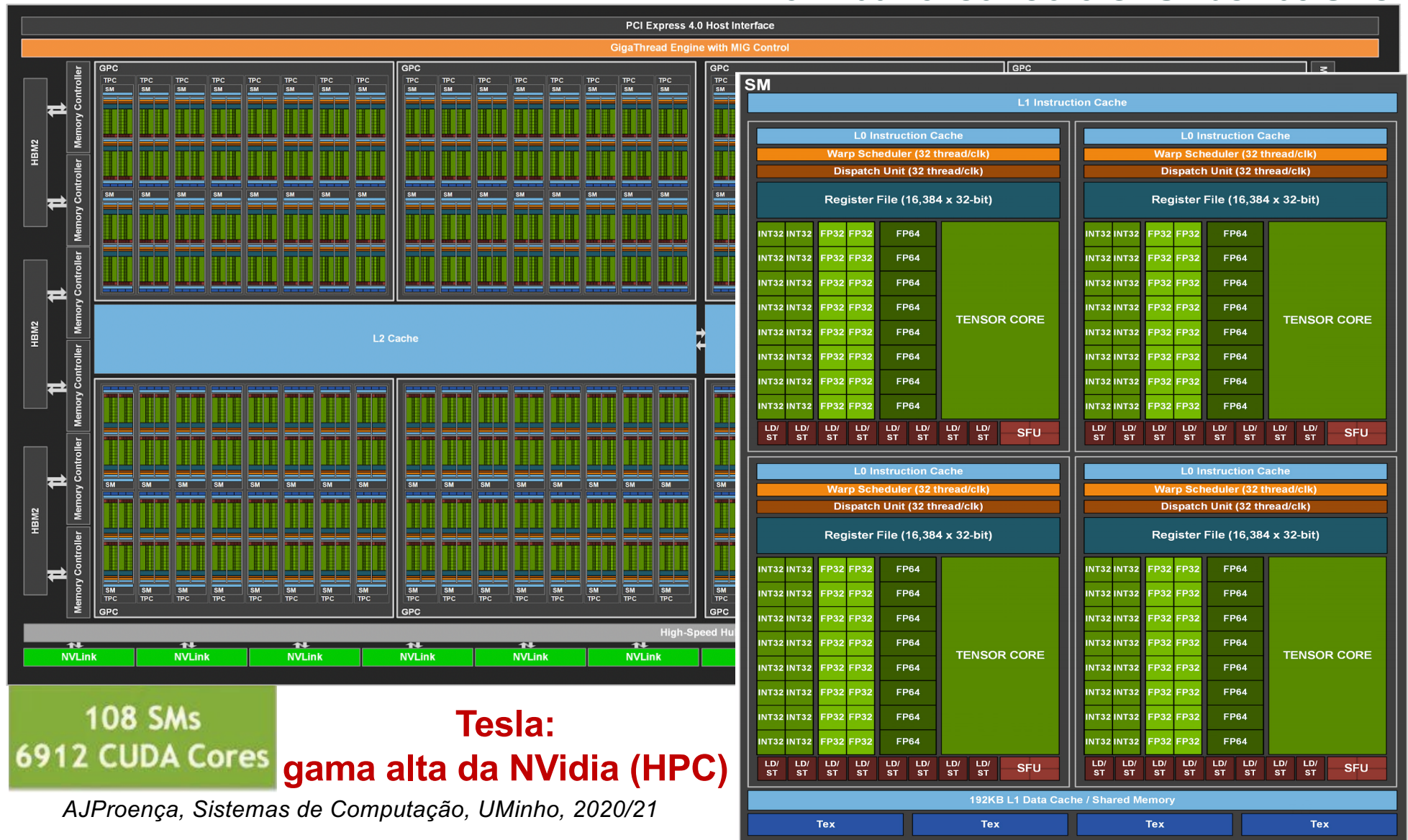


Fermi architecture
July 2011

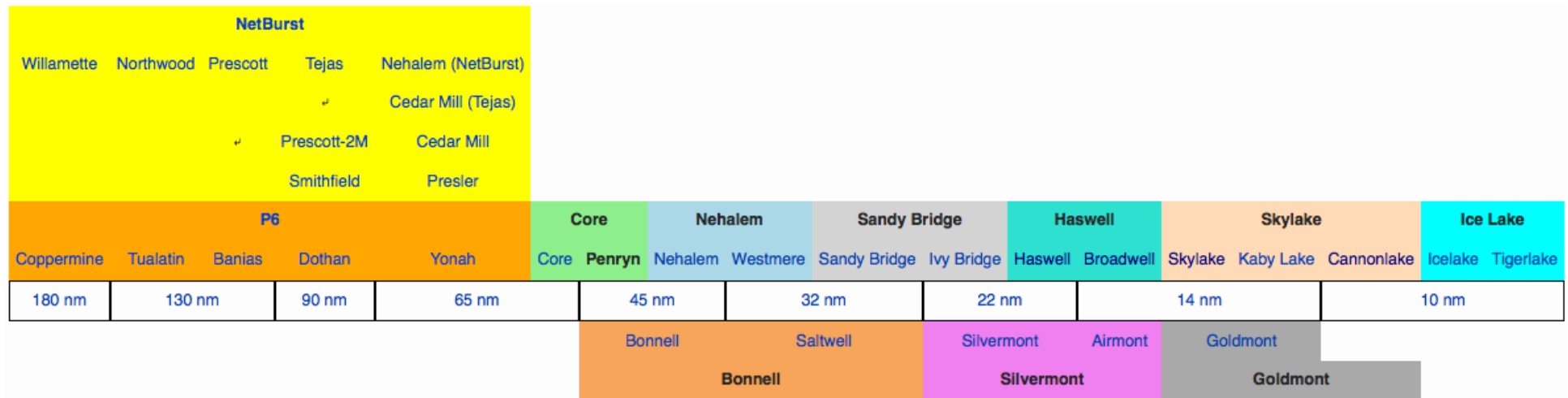
Fermi→Kepler→Maxwell→Pascal→Volta→Ampere: arquitetura do Tesla A100

GA100 full GPU with 128 SMs

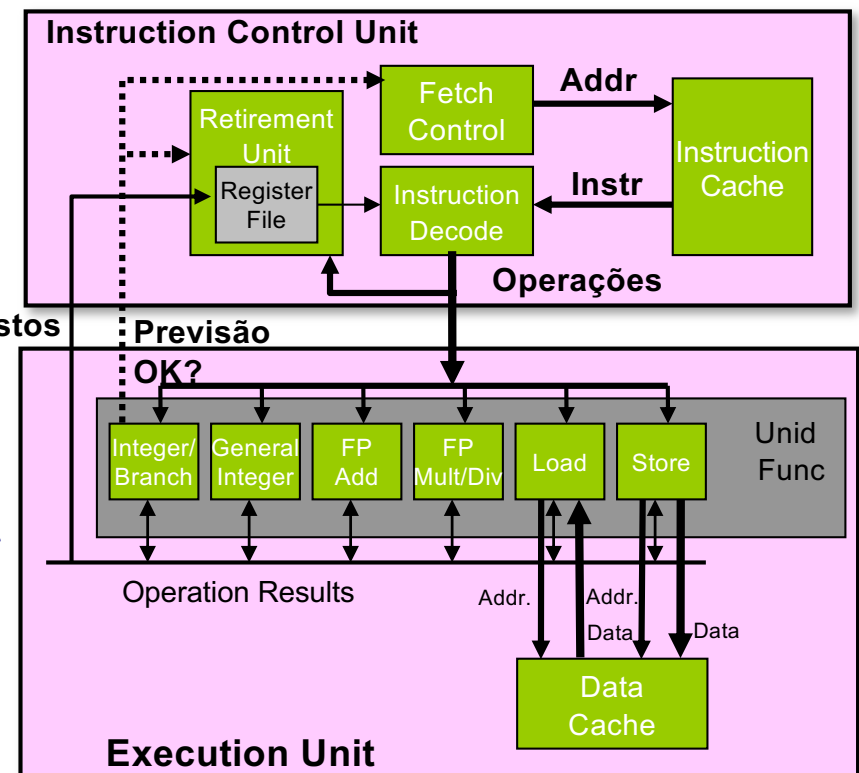
The A100 Tensor Core GPU has 108 SMs



Revisita às microarquiteturas de CPUs da Intel



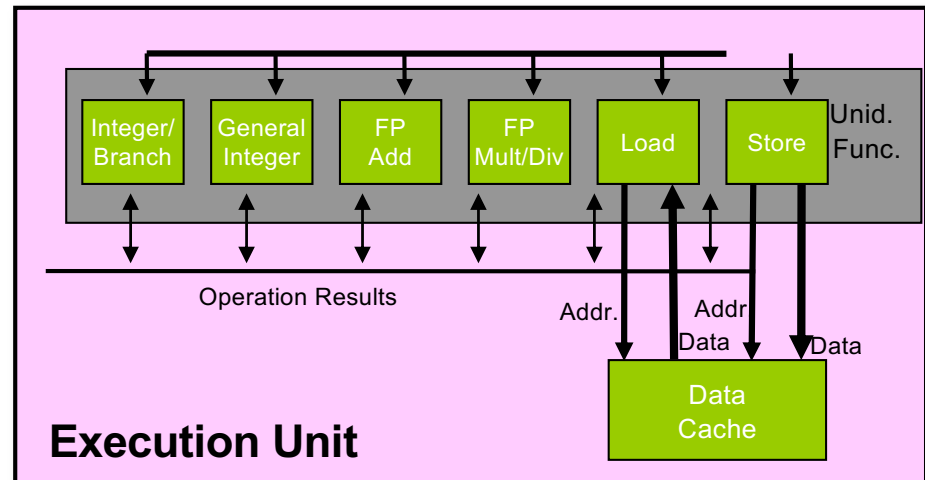
A arquitetura interna dos processadores Intel P6



Algumas potencialidades do Intel P6



- Execução paralela de várias instruções
 - 2 integer (1 pode ser branch)
 - 1 FP Add
 - 1 FP Multiply ou Divide
 - 1 load
 - 1 store



- Algumas instruções requerem > 1 ciclo, mas podem ser encadeadas

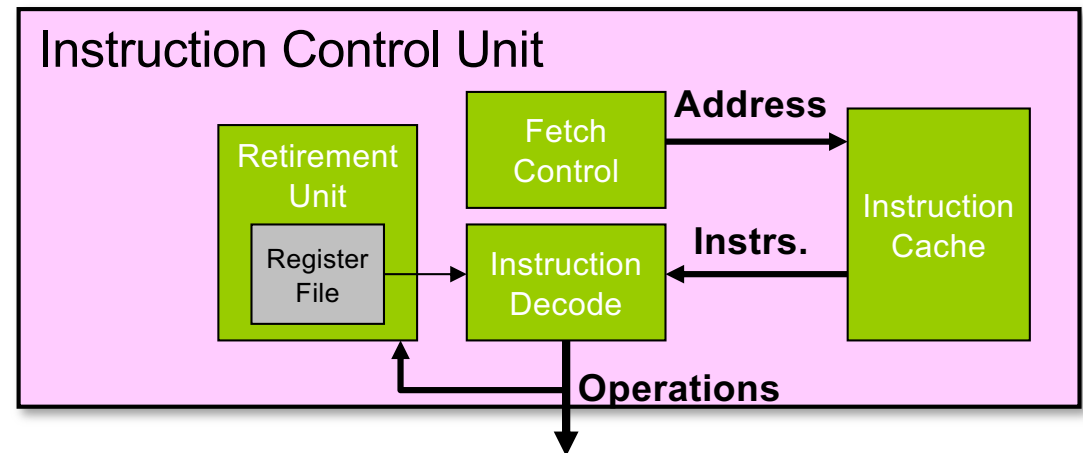
Instrução	Latência	Ciclos/Emissão
Load / Store	3	1
Integer Multiply	4	1
Integer Divide	36	36
Double/Single FP Add	3	1
Double/Single FP Multiply	5	2
Double/Single FP Divide	38	38

A unidade de controlo de instruções do Intel P6



Papel da ICU:

- Lê instruções da *InstrCache*
 - baseado no IP + previsão de saltos
 - antecipa dinamicamente (por *h/w*) se salta/não_salta e (possível) endereço de salto
- Traduz Instruções em *Operações*
 - *Operações*: designação da Intel para instruções tipo-RISC
 - instrução típica do ISA-32 requer 1–3 operações
- Converte referências a Registos em *Tags*
 - *Tags*: identificador abstrato que liga o resultado de uma operação com operandos-fonte de operações futuras



Conversão de instruções com registos para operações com tags



- **Versão de combine4**
 - tipo de dados: *inteiro* ; operação: *multiplicação*

```
.L24:                                # Loop:
    imull (%eax,%edx,4),%ecx         # t *= data[i]
    incl  %edx                       # i++
    cmpl  %esi,%edx                  # i:length
    jl    .L24                       # if < goto Loop
```

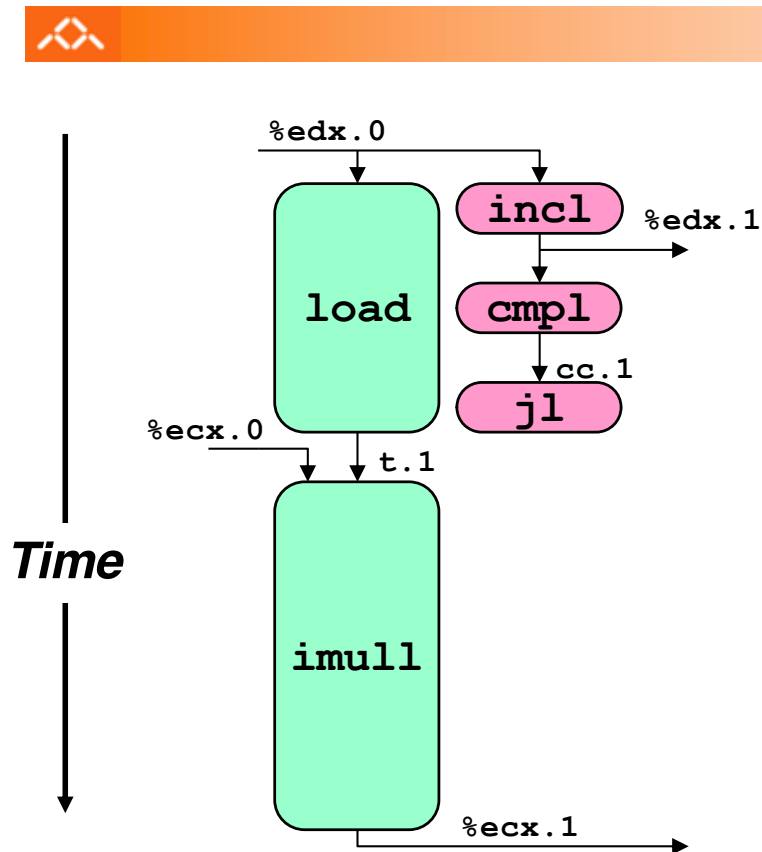
- **Tradução da 1ª iteração**

```
.L24:
    imull (%eax,%edx,4),%ecx

    incl  %edx
    cmpl  %esi,%edx
    jl    .L24
```

```
load (%eax,%edx.0,4)    → t.1
imull t.1, %ecx.0        → %ecx.1
incl  %edx.0             → %edx.1
cmpl  %esi, %edx.1       → cc.1
jl    -taken cc.1
```

Análise visual da execução de instruções no P6: 1 iteração do ciclo de produtos em *combine*



```
load (%eax,%edx.0,4) → t.1
imull t.1, %ecx.0      → %ecx.1
incl %edx.0            → %edx.1
cmpl %esi, %edx.1      → cc.1
j1    -taken cc.1
```

• Operações

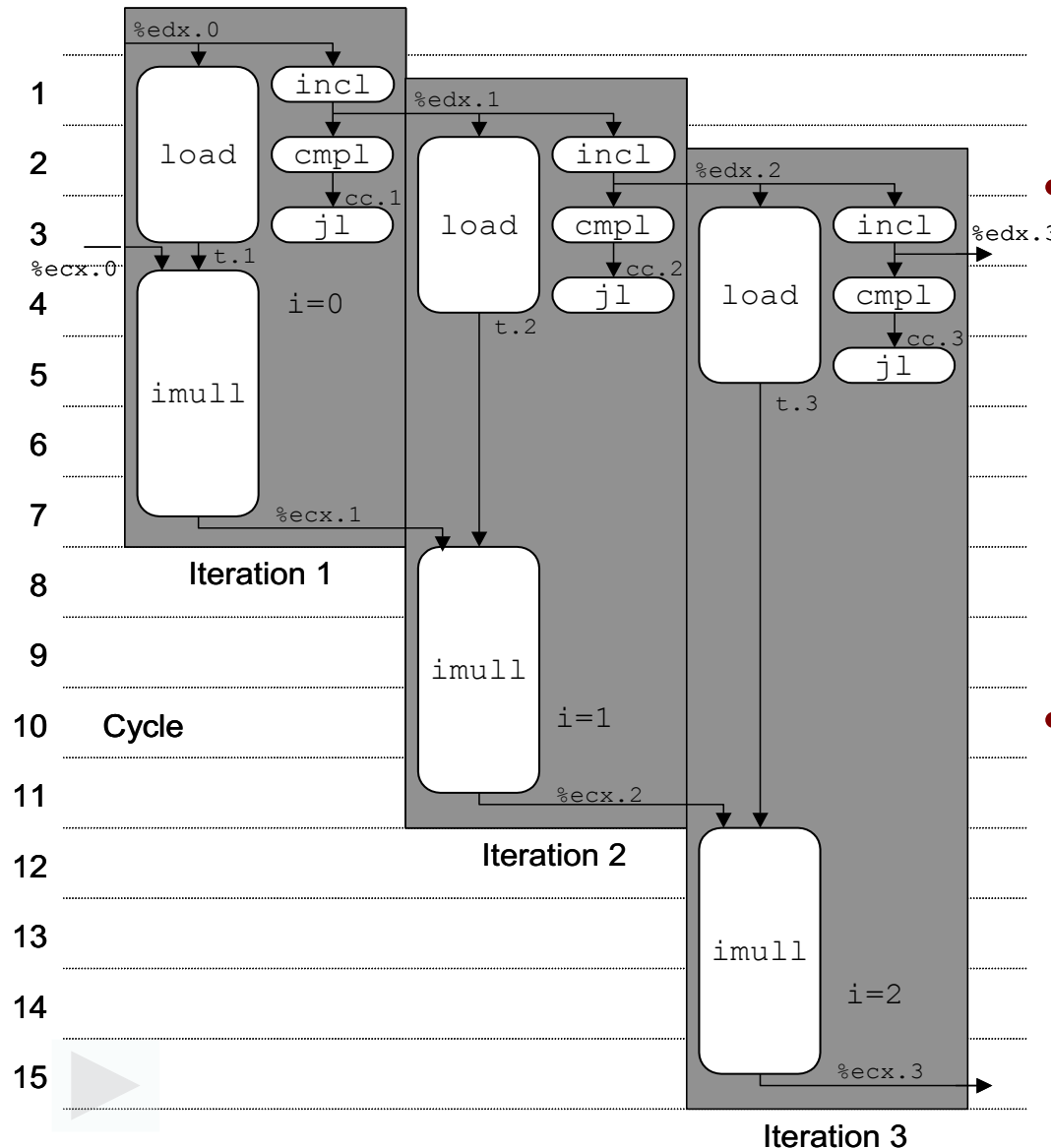
- a posição vertical dá uma indicação do tempo em que é executada
 - uma operação não pode iniciar-se sem os seus operandos
- a altura traduz a latência

• Operandos

- os arcos apenas são representados para os operandos que são usados no contexto da *execution unit*

Instrução	Latência	Ciclos/Emissão
Load / Store	3	1
Integer Multiply	4	1
Integer Divide	36	36
Double/Single FP Add	3	1
Double/Single FP Multiply	5	2
Double/Single FP Divide	38	38

Análise visual da execução de instruções no P6: 3 iterações do ciclo de produtos em *combine*



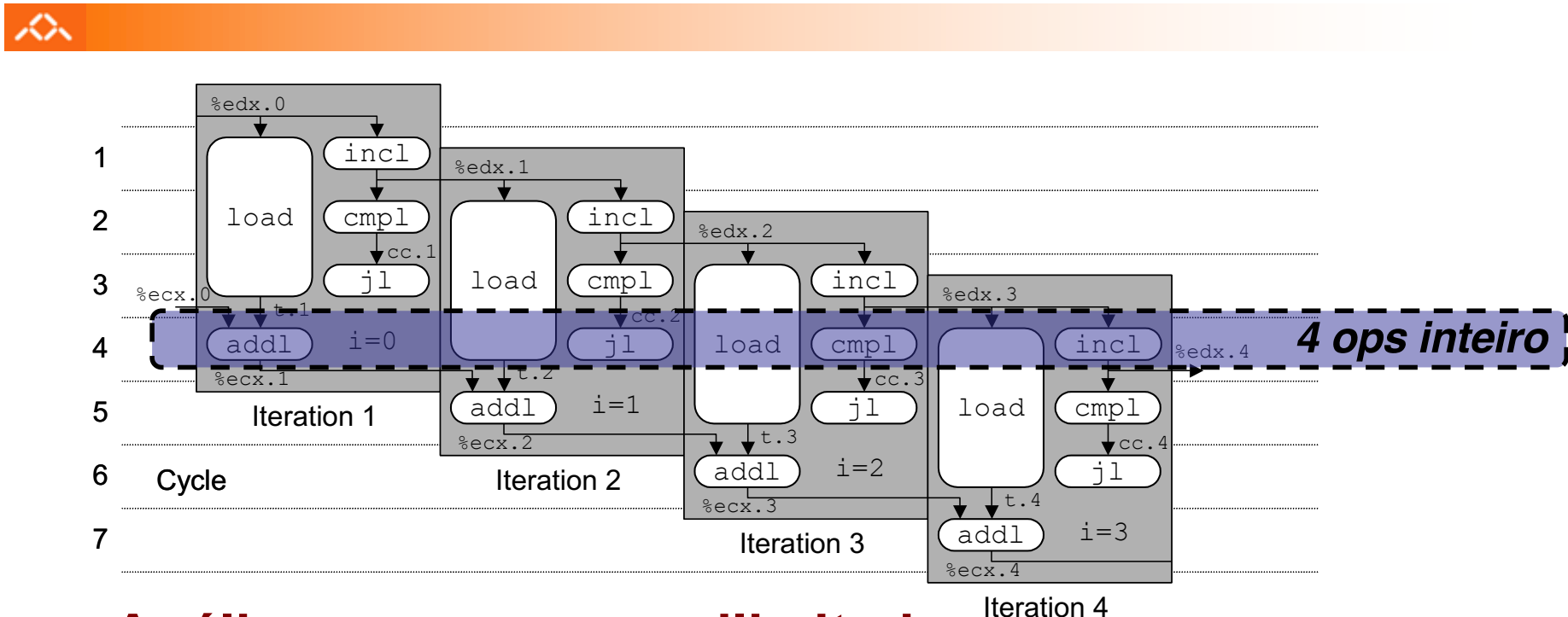
• Análise com recursos ilimitados

- execução paralela e encadeada de operações na EU
- execução *out-of-order* e especulativa

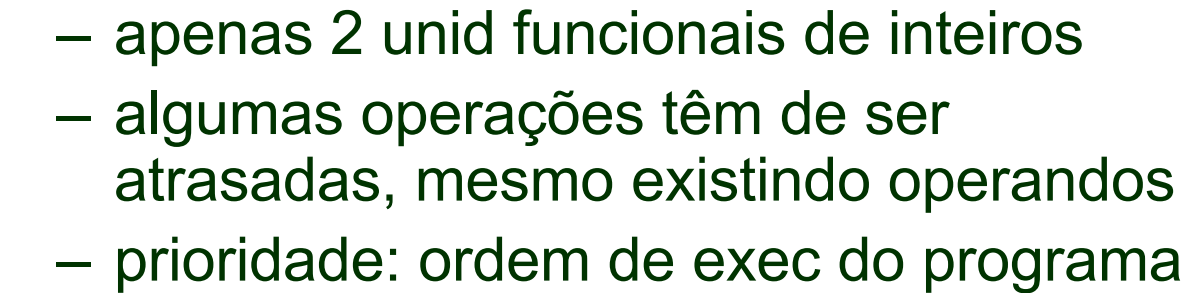
• Desempenho

- factor limitativo: latência da multipl. de inteiros
- CPE: 4.0

Análise visual da execução de instruções no P6: 4 iterações do ciclo de somas em *combine*



- **Análise com recursos ilimitados**
- **Desempenho**
 - pode começar uma nova iteração em cada ciclo de *clock*
 - valor teórico de CPE: 1.0
 - requer a execução de 4 operações c/ inteiros em paralelo



- CPE expectável: 2.0

Avaliação de Desempenho no IA-32 (4)



Estrutura do tema Avaliação de Desempenho (IA-32)

1. A avaliação de sistemas de computação
2. Técnicas de otimização de código (IM)
3. Técnicas de otimização de *hardware*
4. Técnicas de otimização de código (DM)
5. Outras técnicas de otimização
6. Medição de tempos

Análise de técnicas de otimização



Análise de técnicas de otimização (s/w)

- técnicas independentes da máquina ... *já visto...*
- **técnicas de otimização de código (dep. máquina)**
 - análise sucinta de uma PU, Intel x86, P6 (*já visto...*)
 - ***loop unroll*** sem e com paralelismo
 - **identificação de potenciais limitadores de desempenho**
 - ***inline functions***
 - **uso das capacidades de processamento vetorial**
 - dependentes da hierarquia da memória
- **outras técnicas de otimização** (*na próxima aula*)
 - **na compilação:** otimizações efetuadas pelo GCC
 - **na identificação dos "gargalos" de desempenho**
 - *code profiling* e uso dum *profiler* para apoio à otimização
 - **lei de Amdahl**

Técnicas de otimização dependentes da máquina: loop unroll (1)



```
void combine5(vec_ptr v, int *dest)
{
    int length = vec_length(v);
    int limit = length-2;
    int *data = get_vec_start(v);
    int sum = 0;
    int i;
    /* junta 3 elem's no mesmo ciclo */
    for (i = 0; i < limit; i+=3) {
        sum += data[i] + data[i+1]
              + data[i+2];
    }
    /* completa os restantes elem's */
    for (; i < length; i++) {
        sum += data[i];
    }
    *dest = sum;
}
```

Otimização 4:

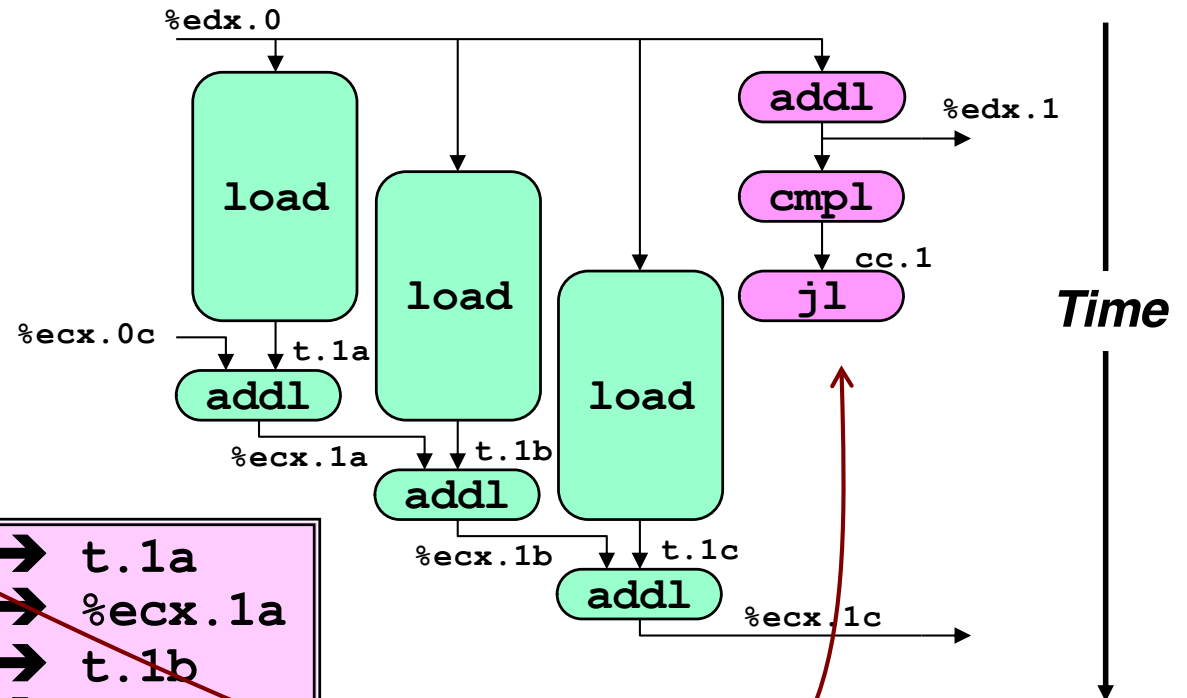
- juntar várias (3) iterações num simples ciclo
- amortiza *overhead* dos ciclos em várias iterações
- termina extras no fim
- **CPE: 1.33**

Técnicas de otimização dependentes da máquina: loop unroll (2)

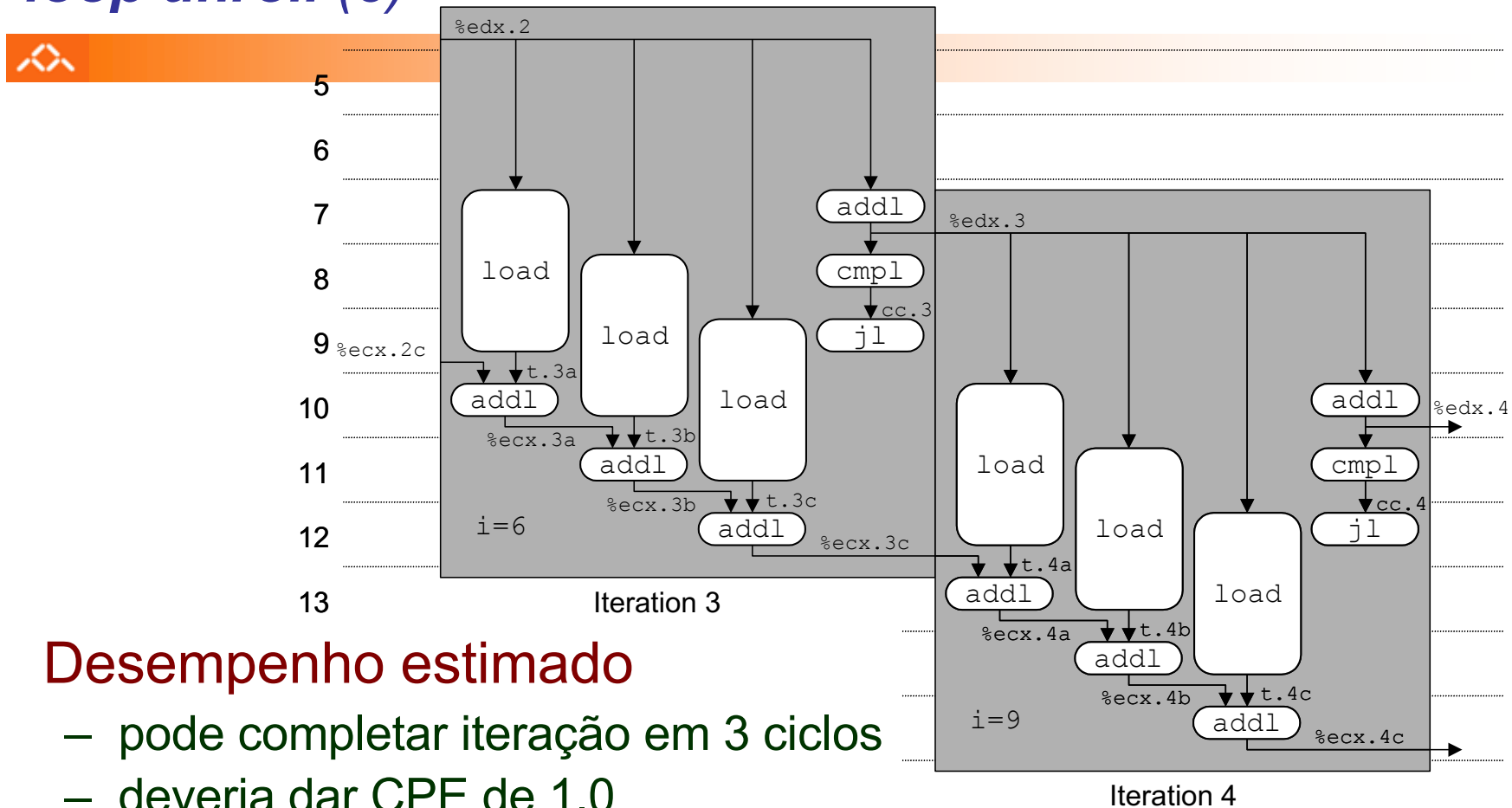


- **loads** podem encadear, uma vez que não há dependências
- apenas um conjunto de instruções de controlo de ciclo

```
load (%eax,%edx.0,4)  ➔ t.1a
iaddl t.1a, %ecx.0c    ➔ %ecx.1a
load 4(%eax,%edx.0,4) ➔ t.1b
iaddl t.1b, %ecx.1a    ➔ %ecx.1b
load 8(%eax,%edx.0,4) ➔ t.1c
iaddl t.1c, %ecx.1b    ➔ %ecx.1c
iaddl $3,%edx.0        ➔ %edx.1
cmpl %esi, %edx.1      ➔ cc.1
jl  -taken cc.1
```



Técnicas de otimização dependentes da máquina: loop unroll (3)



- **Desempenho estimado**
 - pode completar iteração em 3 ciclos
 - deveria dar CPE de 1.0
- **Desempenho medido**
 - CPE: 1.33
 - 1 iteração em cada 4 ciclos

Técnicas de otimização dependentes da máquina: *loop unroll* (4)



Valor do **CPE** para várias situações de *loop unroll*:

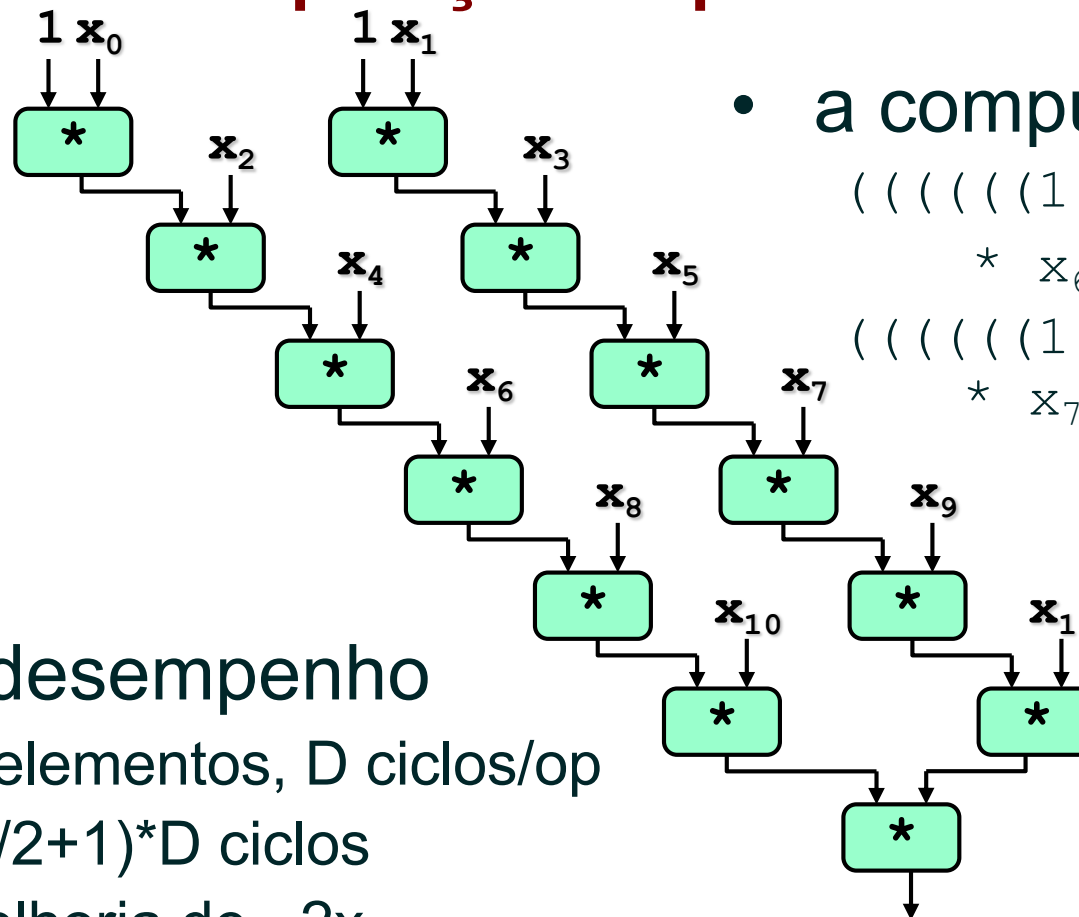
Grau de <i>Unroll</i>		1	2	3	4	8	16
Inteiro	Soma	2.00	1.50	1.33	1.50	1.25	1.06
Inteiro	Produto	4.00					
<i>fp</i>	Soma	3.00					
<i>fp</i>	Produto	5.00					

- apenas melhora nas somas de inteiros
 - restantes casos há restrições com a latência da unidade
- efeito não é linear com o grau de *unroll*
 - há efeitos subtis que determinam a atribuição exacta das operações

Técnicas de otimização dependentes da máquina: ... versus computação paralela



Computação sequencial ... versus paralela!



- a computação...

$$(((((((1 * x_0) * x_2) * x_4) * x_6) * x_8) * x_{10}) * ((((((1 * x_1) * x_3) * x_5) * x_7) * x_9) * x_{11}))$$

... o desempenho

- N elementos, D ciclos/op
- (N/2+1)*D ciclos
- melhoria de ~2x

Técnicas de otimização dependentes da máquina: loop unroll com paralelismo (1)



```
void combine6(vec_ptr v, int *dest)
{
    int length = vec_length(v);
    int limit = length-1;
    int *data = get_vec_start(v);
    int x0 = 1;
    int x1 = 1;
    int i;
    /* junta 2 elem's de cada vez */
    for (i = 0; i < limit; i+=2) {
        x0 *= data[i];
        x1 *= data[i+1];
    }
    /* completa os restantes elem's */
    for (; i < length; i++) {
        x0 *= data[i];
    }
    *dest = x0 * x1;
}
```

... versus paralela!

Otimização 5:

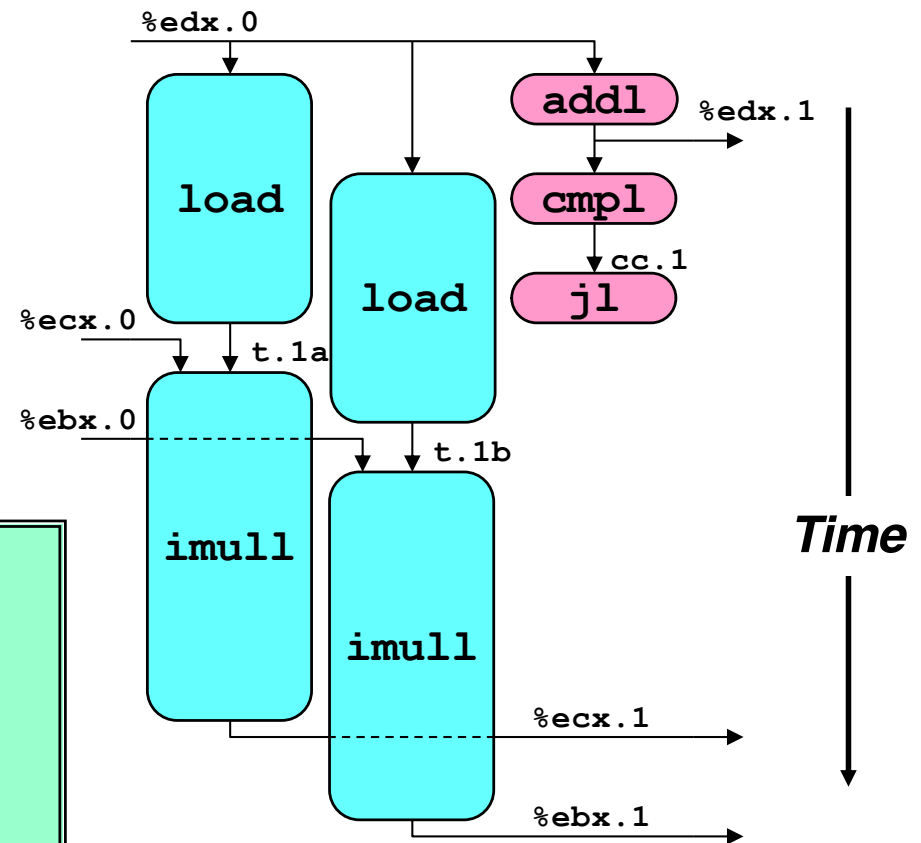
- acumular em 2 produtos diferentes
 - pode ser feito em paralelo, se OP fôr associativa!
- juntar no fim
- Desempenho
 - CPE: 2.0
 - melhoria de 2x

Técnicas de otimização dependentes da máquina: loop unroll com paralelismo (2)

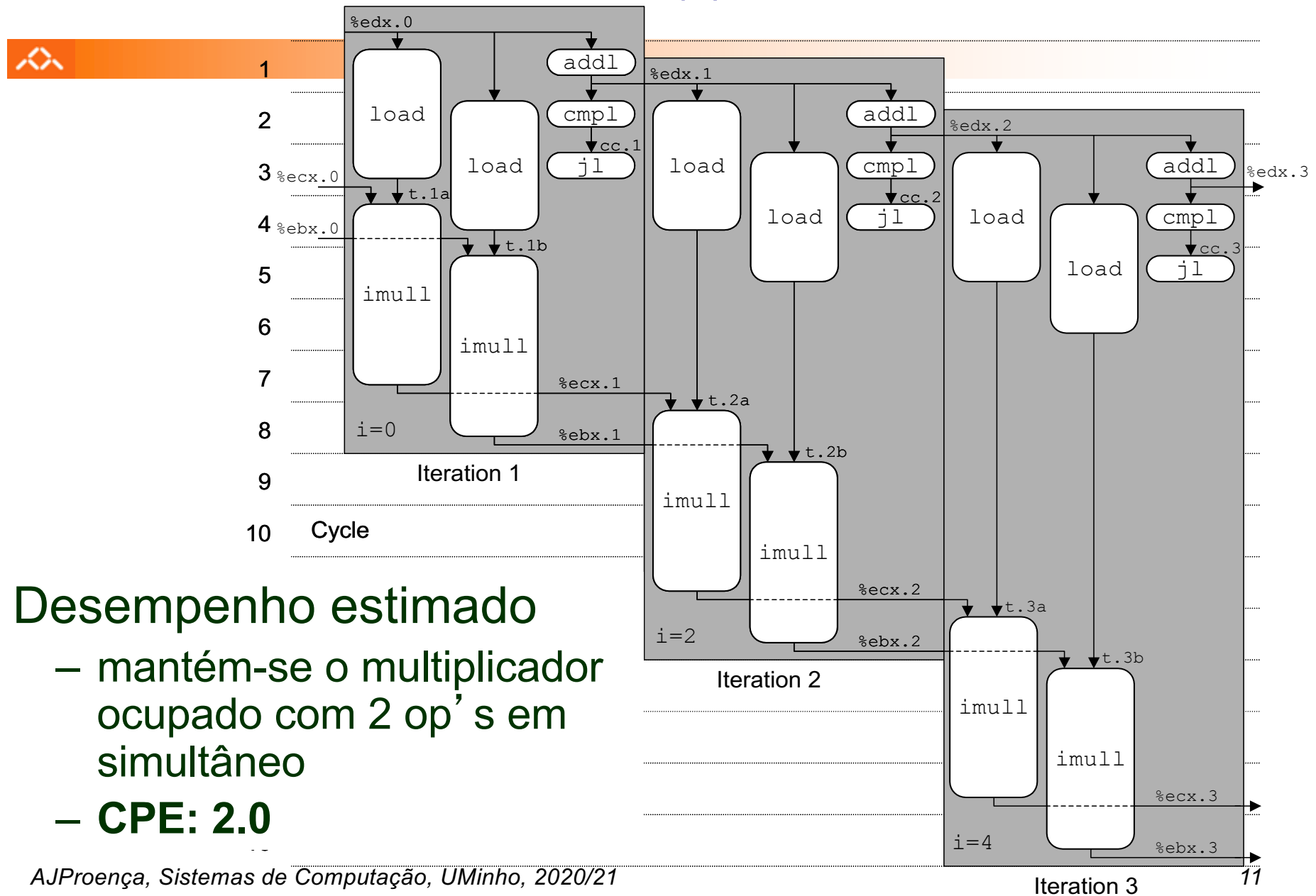


- os dois produtos no interior do ciclo não dependem um do outro...
- e é possível encadeá-los
- *iteration splitting*, na literatura

```
load (%eax,%edx.0,4)  → t.1a
imull t.1a, %ecx.0     → %ecx.1
load 4(%eax,%edx.0,4) → t.1b
imull t.1b, %ebx.0     → %ebx.1
iaddl $2,%edx.0       → %edx.1
cmpl %esi, %edx.1     → cc.1
jl-taken cc.1
```



Técnicas de otimização dependentes da máquina: loop unroll com paralelismo (3)



Técnicas de otimização de código: análise comparativa de *combine*



Método	Inteiro		Real (precisão simples)	
	+	*	+	*
Abstract -g	42.06	41.86	41.44	160.00
Abstract -O2	31.25	33.25	31.25	143.00
Move vec_length	20.66	21.25	21.15	135.00
Acesso aos dados	6.00	9.00	8.00	117.00
Acum. em temp	2.00	4.00	3.00	5.00
Unroll 4x	1.50	4.00	3.00	5.00
Unroll 16x	1.06	4.00	3.00	5.00
Unroll 2x, paral. 2x	1.50	2.00	2.00	2.50
Unroll 4x, paral. 4x	1.50	2.00	1.50	2.50
Unroll 8x, paral. 4x	1.25	1.25	1.50	2.00
Otimização Teórica	1.00	1.00	1.00	2.00
Rácio Pior : Melhor	39.7	33.5	27.6	80.0

Otimização de código: limitações do paralelismo ao nível da instrução



- Precisa de muitos registos!
 - para guardar somas/produtos
 - apenas 6 registos (p/ inteiros) disponíveis no IA-32
 - tb usados como apontadores, controlo de ciclos, ...
 - 8 registos de fp
 - quando os registos são insuficientes, temp's vão para a *stack*
 - elimina ganhos de desempenho
(ver *assembly* em produto inteiro com *unroll* 8x e paralelismo 8x)
 - re-nomeação de registos não chega
 - não é possível referenciar mais operandos que aqueles que o *instruction set* permite
 - ... principal inconveniente do *instruction set* do IA-32
- Operações a paralelizar têm de ser associativas
 - a soma e multipl de fp num computador não é associativa!
 - $(3.14+1e20)-1e20$ nem sempre é igual a $3.14+(1e20-1e20)$...

Limitações do paralelismo: a insuficiência de registos



- **combine**

- produto de inteiros
- *unroll* 8x e paralelismo 8x
- 7 variáveis locais partilham 1 registo (**%edi**)
 - observar os acessos à *stack*
 - melhoria desempenho é comprometida...
 - *register spilling* na literatura

```
.L165:
    imull (%eax), %ecx
    movl -4(%ebp), %edi
    imull 4(%eax), %edi
    movl %edi, -4(%ebp)
    movl -8(%ebp), %edi
    imull 8(%eax), %edi
    movl %edi, -8(%ebp)
    movl -12(%ebp), %edi
    imull 12(%eax), %edi
    movl %edi, -12(%ebp)
    movl -16(%ebp), %edi
    imull 16(%eax), %edi
    movl %edi, -16(%ebp)
    ...
    addl $32, %eax
    addl $8, %edx
    cmpl -32(%ebp), %edx
    jl .L165
```

Execução de funções inline



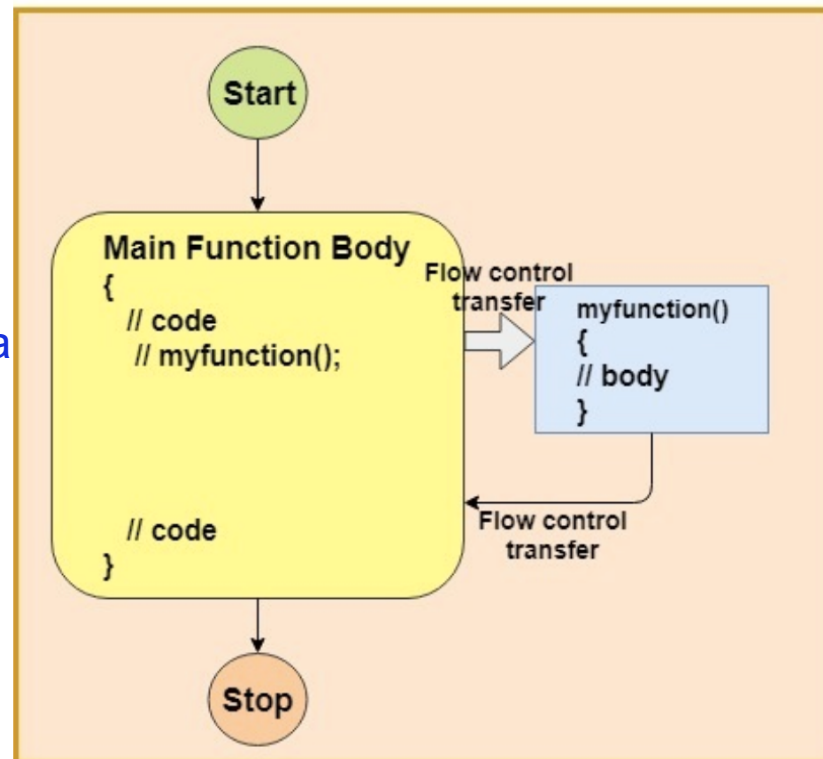
WIKIPEDIA

In **computing**, **inline expansion**, or **inlining**, is a manual or **compiler optimization** that replaces a function **call site** with the body of the called function. Inline expansion is similar to **macro expansion**, but

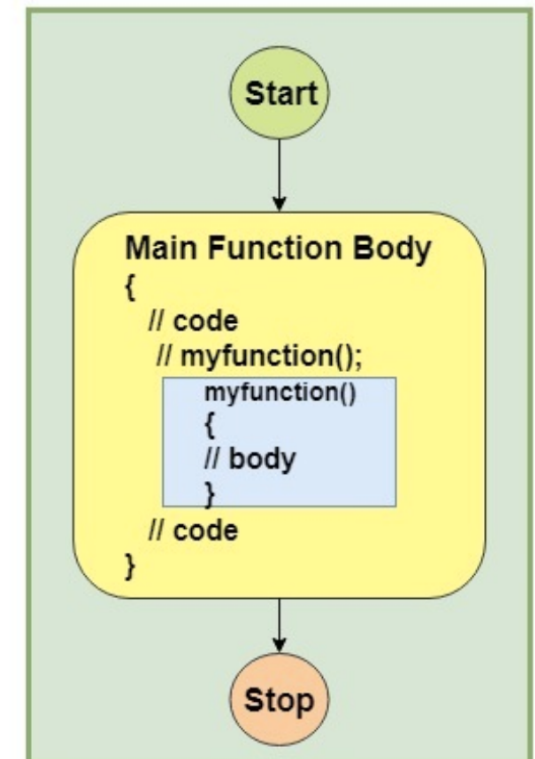
Funções *inline*:

- pode compensar se a função tiver um corpo (*body*) pequeno
- poupa a sobrecarga de invocar uma função

Normal Function



Inline Functions



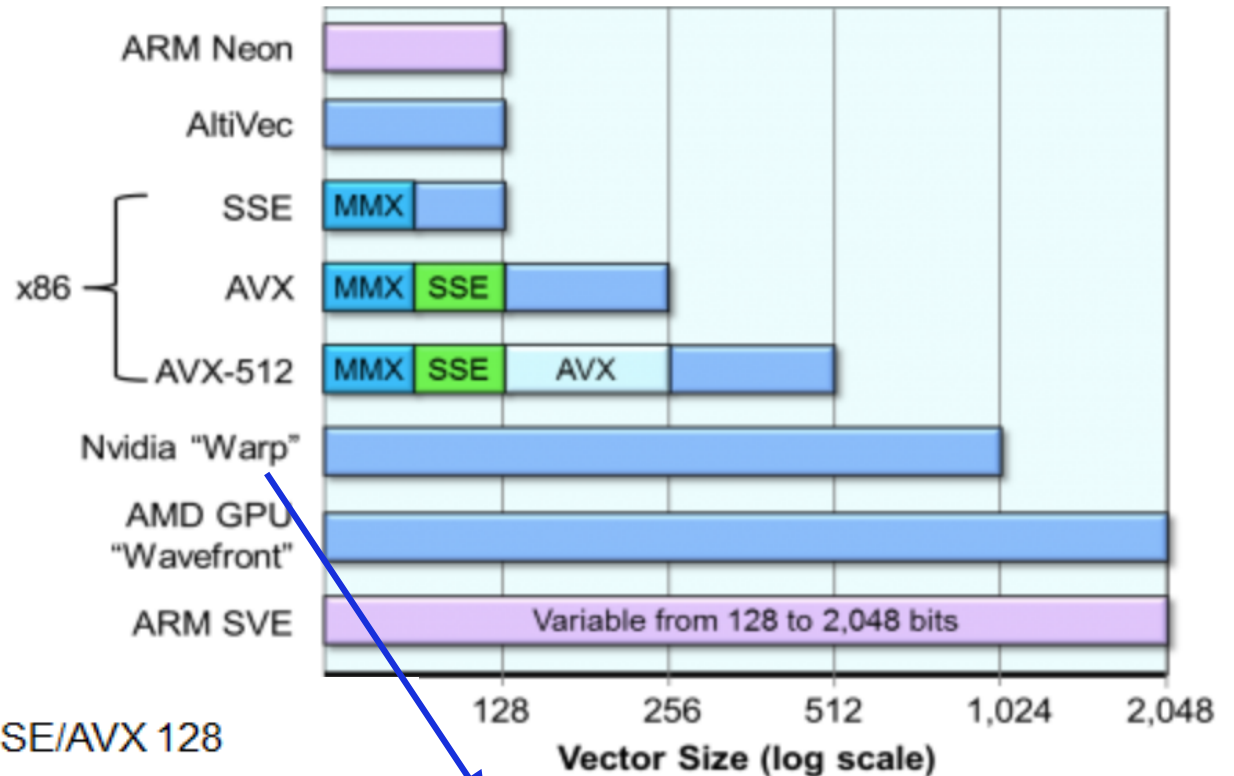
Explorar o paralelismo de dados



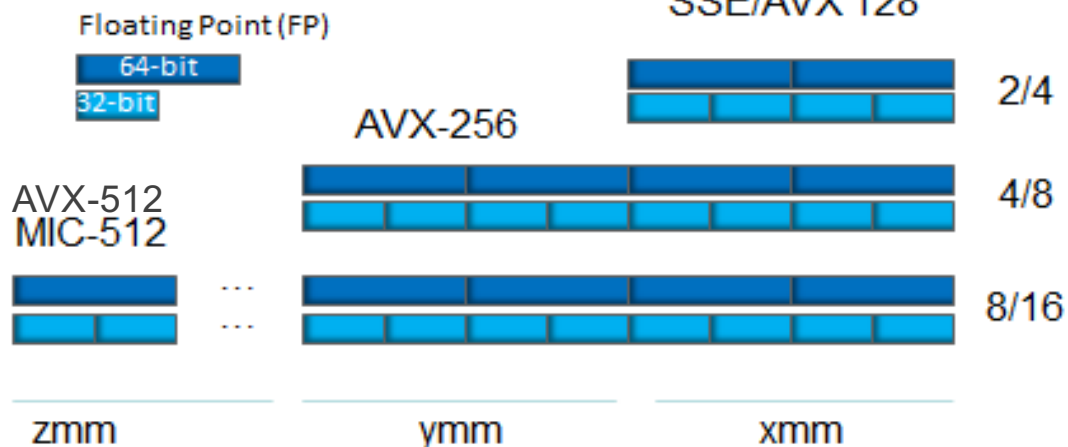
Alternativas:

- PU vetorial
ex: Cray 1, GPUs
- extensões vetoriais
às PUs escalares

ISA-extensions



Intel ISA-extensions



Warp: bloco de 32 *threads* que é executado ao mesmo tempo com a mesma instrução com `int` ou `float` => **SIMT**, vetor de 32*32 bits

Avaliação de Desempenho no IA-32 (5)



Estrutura do tema Avaliação de Desempenho (IA-32)

1. A avaliação de sistemas de computação
2. Técnicas de otimização de código (IM)
3. Técnicas de otimização de *hardware*
4. Técnicas de otimização de código (DM)
5. Outras técnicas de otimização
6. Medição de tempos

Análise de técnicas de otimização



Análise de técnicas de otimização (s/w)

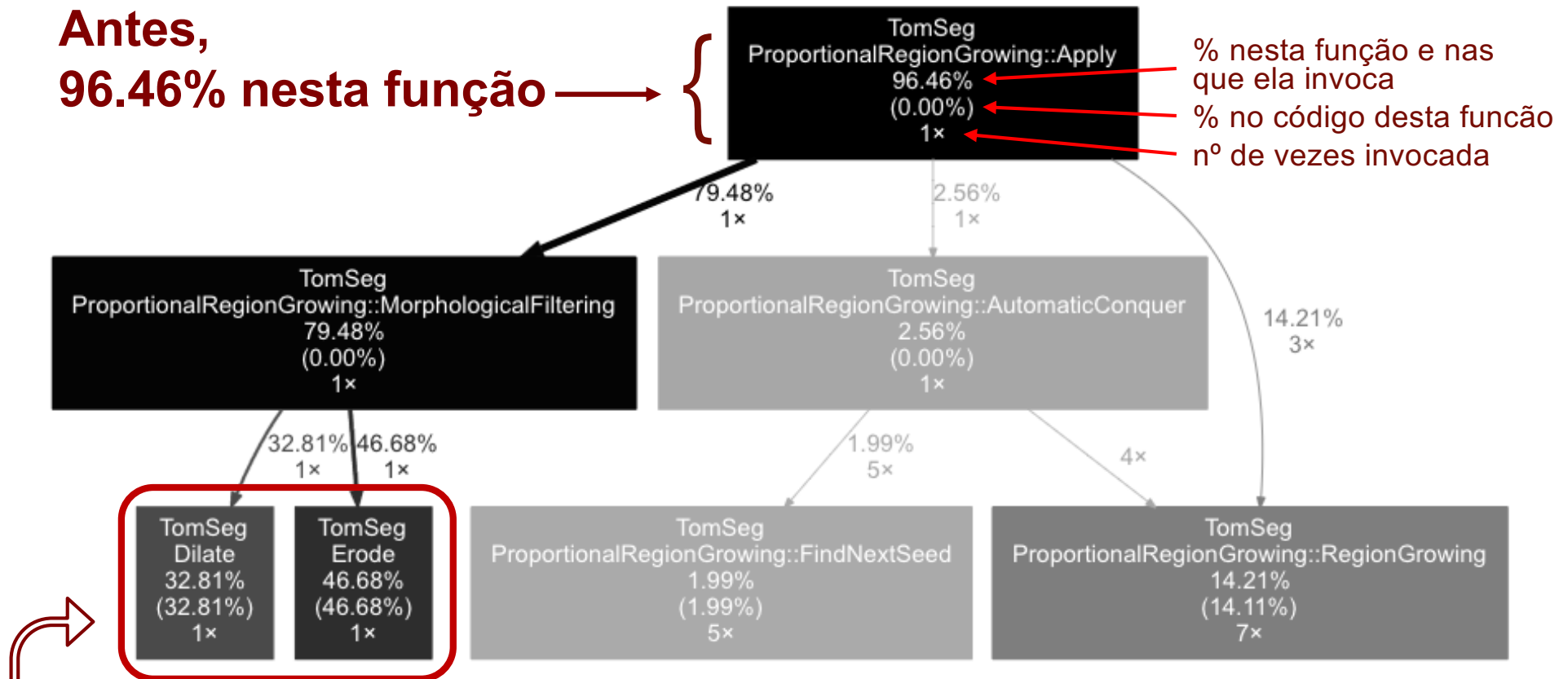
- técnicas de otimização de código (independ. máquina)
 - *já visto...*
- técnicas de otimização de código (depend. máquina)
 - dependentes do processador (*já visto...*)
 - dependentes da hierarquia da memória (*a ver numa próxima UC*)
 - a localidade espacial e temporal dum programa (*já visto...*)
 - quantificação da influência da *cache* no desempenho
- **outras técnicas de otimização**
 - na compilação: otimizações efetuadas pelo GCC
 - na identificação dos "gargalos" de desempenho
 - *code profiling*
 - uso dum *profiler* para apoio à otimização
- **lei de Amdahl**

Code profiling: análise visual da melhoria de código numa função (antes)



Antes,

96.46% nesta função →



Quase 80% do tempo total é gasto nestas 2 funções da filtragem morfológica!
Conclusão: é aqui que se deve investir para melhorar a *performance* global

Figure 5.7.: Call-graph of the first version of *Propor. Region Growing* (DS3)

Code profiling: análise visual da melhoria de código duma função (depois)

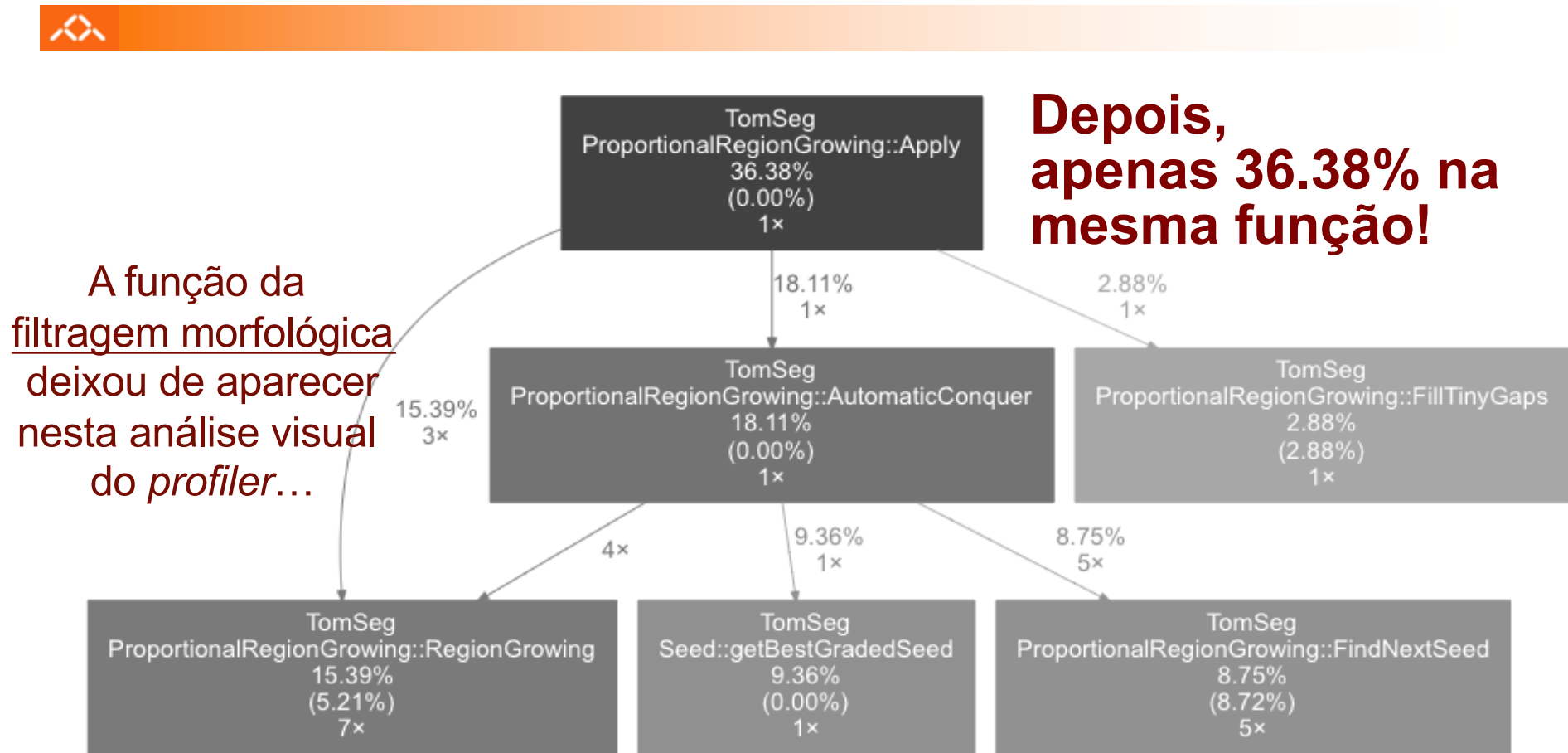


Figure 5.9.: Call-graph from the last version of *Propor. Region Growing* (DS3)

Code profiling e visualização: dicas para a sua realização



Mais comum para aplicações em ambiente GNU/Linux:

- `gprof` : disponível em `gcc/binutils`
- `gprof` requer compilação com `-pg` para gerar o ficheiro `gmon.out` contendo os dados para o *profiler*
- `gprof` lê a informação em `gmon.out` e produz um relatório em `main.gprof`
- `gprof2dot` lê `main.gprof` e gera uma representação visual dos dados tal como apresentado nos slides anteriores
- `gprof2dot` disponível em <https://github.com/jrfonseca/gprof2dot>
- para visualizar a imagem do *call graph* usar o comando `xdot`

https://developer.ridgerun.com/wiki/index.php/Profiling_GNU/Linux_applications

Lei de Amdahl



O ganho no desempenho – *speedup* –

obtido com a melhoria do tempo de execução de uma parte do sistema, está limitado pela fração de tempo que essa parte do sistema pode ser usada.

$$\text{Speedup}_{\text{overall}} = \frac{\text{Tempo_exec}_{\text{antigo}}}{\text{Tempo_exec}_{\text{novo}}} = \frac{1}{\sum (f_i / s_i)}$$

f_i - fracções com melhoria s_i
 s_i - *speedup* de cada fracção

Ex.1: Se 10% de um programa executa 90x mais rápido

Overall speedup = 1.11

Ex.2: Se 90% de um prog executa 90x mais rápido

Overall speedup = 9.09

Paralelismo:
se $N_{\text{proc}} \equiv \text{speedup}$, trocar s_i por N_{proc}

